MODULE 1: CORE BACKEND AND API

FILE: backend/main.py [Lines 75–205 of 2261]

```
    allow_methods=["*"],
    allow_headers=["*"],
)

# ── REQUEST/RESPONSE SCHEMAS ──────────────────────────────────────

class LoginRequest(BaseModel):
    username: str
    password: str


class PrescriptionCreateRequest(BaseModel):
    patient_id: int
    medicine_id: int
    dosage: str
    frequency: str = "Every 8 hours"
    frequency_hours: int = 8
    route: str = "Oral"
    duration_days: int = 7
    instructions: Optional[str] = None


class DiscontinuePrescriptionRequest(BaseModel):
    reason: str


class CreateDeliveryTaskRequest(BaseModel):
    patient_id: int
    medicine_id: int
    priority: str = "NORMAL"  # LOW, NORMAL, HIGH, EMERGENCY
    prescription_id: Optional[int] = None


class TaskTransitionRequest(BaseModel):
    status: str
    message: Optional[str] = None
    progress_pct: Optional[float] = None


class VoiceCommandRequest(BaseModel):
    command: str


class CameraControlRequest(BaseModel):
    mode: str
    target_robot_id: Optional[str] = None


class EmergencyTriggerRequest(BaseModel):
    event_type: str = "CODE_BLUE"
    code_type: Optional[str] = None
    room_number: str = "ICU-01"
    details: Optional[str] = "Critical patient cardiopulmonary emergency."
    description: Optional[str] = None


class ExplorationRequest(BaseModel):
    robot_id: str = "Arogya_Delivery"
    mode: str = "FRONTIER_EXPLORATION"


class ObstacleToggleRequest(BaseModel):
    location: str = "MAIN_CORRIDOR"
    x: float = 0.0
    y: float = 0.0
    radius: float = 0.6


class FleetDispatchRequest(BaseModel):
    mission_type: str = "MEDICINE_DELIVERY"
    destination: str = "ICU-01"
    patient_id: Optional[str] = "P001"
    medicine_name: Optional[str] = "Paracetamol 500mg"
    priority: Optional[str] = "NORMAL"
    preferred_robot_id: Optional[str] = "AUTO"
```

```
    robot_id: Optional[str] = None
    evaluate_only: Optional[bool] = False
    notes: Optional[str] = None


class FleetCancelRequest(BaseModel):
    robot_id: str = "ROBOT-01"
    task_code: Optional[str] = None
    reason: Optional[str] = "Operator manual override"


class FleetReassignRequest(BaseModel):
    current_robot_id: Optional[str] = None
    source_robot_id: Optional[str] = None
    target_robot_id: Optional[str] = "AUTO"
    task_code: Optional[str] = None
    reason: Optional[str] = "Dynamic load balancing"


class FleetChargeRequest(BaseModel):
    robot_id: str = "ROBOT-01"
    dock_id: Optional[str] = "CHG-01"


class RobotControlRequest(BaseModel):
    robot_id: Optional[str] = "Arogya_Delivery"
    action: str = "stop"  # forward, reverse, left, right, stop
    linear_speed: Optional[float] = 0.4
    angular_speed: Optional[float] = 0.8


# ── 0. SYSTEM HEALTH & DIAGNOSTICS (Requirement 2) ──────────────────────

@app.get("/api/health")
@app.get("/api/system/health")
def get_system_health(db: Session = Depends(get_db)):
    """
    Real System Health Check verifying all subsystems (Requirement 2):
    Backend, Database, Authentication, Agent system, ROS 2, Isaac Sim,
    Robot fleet, Navigation, Telemetry.
    """
    backend_status = {
        "status": "ONLINE",
        "version": "3.0.0",
        "ready": True,
    }

    try:
        patient_count = db.query(Patient).count()
        user_count = db.query(User).count()
        room_count = db.query(HospitalRoom).count()
        robot_count = db.query(Robot).count()
        db_status = {
            "status": "ONLINE",
            "ready": True,
            "details": {
```

FILE: backend/medication_engine.py [Lines 25–145 of 192]

```
    ScheduleStatusEnum,
)


class MedicationValidationResult:
    def __init__(
        self,
        is_valid: bool,
        reason: str,
        prescription: Optional[Prescription] = None,
        patient: Optional[Patient] = None,
        medicine: Optional[Medicine] = None,
    ) -> None:
        self.is_valid = is_valid
        self.reason = reason
        self.prescription = prescription
        self.patient = patient
        self.medicine = medicine

    def to_dict(self) -> Dict[str, Any]:
        return {
            "is_valid": self.is_valid,
            "reason": self.reason,
```

```
            "prescription_code": self.prescription.prescription_code if self.prescription else None,
            "patient_name": self.patient.name if self.patient else None,
            "medicine_name": self.medicine.name if self.medicine else None,
        }


class MedicationEngine:
    @staticmethod
    def validate_medication_delivery(
        db: Session,
        patient_id: int,
        medicine_id: Optional[int] = None,
        prescription_id: Optional[int] = None,
    ) -> MedicationValidationResult:
        """
        Executes Section 15 & 26 Safety Pre-Conditions:
        - Prescription active?
        - Patient still admitted & active?
        - Prescription discontinued or changed?
        - Patient discharged?
        - Medicine available and in stock?
        """
        # 1. Verify Patient
        patient = db.query(Patient).filter(Patient.id == patient_id).first()
        if not patient:
            return MedicationValidationResult(False, f"Patient ID {patient_id} does not exist in hospital database.")

        if patient.status == PatientStatusEnum.DISCHARGED:
            return MedicationValidationResult(
                False,
                f"SAFETY BLOCK: Patient {patient.name} ({patient.patient_code}) was DISCHARGED on
                    {patient.discharge_date}. Delivery denied.",
                patient=patient,
            )

        if patient.status not in (PatientStatusEnum.ACTIVE, PatientStatusEnum.EMERGENCY):
            return MedicationValidationResult(
                False,
                f"SAFETY BLOCK: Patient status is '{patient.status.value}'. Must be ACTIVE or EMERGENCY for
                    medication delivery.",
                patient=patient,
            )

        if not patient.room_id and not patient.bed_id:
            return MedicationValidationResult(
                False,
                f"Patient {patient.name} is not assigned to any hospital room or bed.",
                patient=patient,
            )

        # 2. Find and Validate Prescription
        rx_query = db.query(Prescription).filter(Prescription.patient_id == patient.id)
        if prescription_id:
            rx_query = rx_query.filter(Prescription.id == prescription_id)
        elif medicine_id:
            rx_query = rx_query.filter(Prescription.medicine_id == medicine_id)

        prescriptions = rx_query.all()
        if not prescriptions:
            return MedicationValidationResult(
                False,
                f"CLINICAL SAFETY BLOCK: No prescription on file for Patient {patient.name} matching requested
                    medicine.",
                patient=patient,
            )

        # Check for active prescription
        active_rx = None
        for rx in prescriptions:
            if rx.status == PrescriptionStatusEnum.DISCONTINUED:
                return MedicationValidationResult(
                    False,
                    f"CLINICAL SAFETY BLOCK: Prescription {rx.prescription_code} for {rx.medicine.name} was
                        DISCONTINUED by attending physician. Cannot deliver.",
                    prescription=rx,
                    patient=patient,
                    medicine=rx.medicine,
                )
            elif rx.status == PrescriptionStatusEnum.PAUSED:
                return MedicationValidationResult(
                    False,
                    f"Prescription {rx.prescription_code} is currently PAUSED by clinical staff.",
                    prescription=rx,
```

```
                patient=patient,
                medicine=rx.medicine,
            )
        elif rx.status == PrescriptionStatusEnum.COMPLETED:
            return MedicationValidationResult(
                False,
                f"Prescription {rx.prescription_code} treatment course has been COMPLETED.",
                prescription=rx,
                patient=patient,
                medicine=rx.medicine,
            )
        elif rx.status == PrescriptionStatusEnum.ACTIVE:
            active_rx = rx
            break

    if not active_rx:
        return MedicationValidationResult(
            False,
            f"No ACTIVE prescription found for patient {patient.name}.",
```

MODULE 2: MULTI-AGENT ARCHITECTURE

FILE: agents/agent_base.py [Lines 40–195 of 288]

```
# ── Agent States ──────────────────────────────────────────────────────

class AgentState(Enum):
    """Operational state of a hospital agent."""
    IDLE       = auto()   # Available for new tasks
    BIDDING    = auto()   # Evaluating a contract-net call-for-proposals
    ASSIGNED   = auto()   # Task assigned, preparing execution
    EXECUTING  = auto()   # Actively running a mission
    RETURNING  = auto()   # Returning to dock / home position
    CHARGING   = auto()   # At dock, charging
    ERROR      = auto()   # Requires operator attention
    OFFLINE    = auto()   # Not responding


# ── Agent Bid ─────────────────────────────────────────────────────────

@dataclass
class AgentBid:
    """A bid submitted by an agent in response to a task broadcast.

    Attributes:
        agent_id:       Unique agent identifier.
        score:          Composite fitness score (0.0–1.0, higher is better).
        estimated_eta:  Estimated seconds to complete the mission.
        battery_pct:    Agent's current battery percentage.
        distance:       Grid distance from agent to first task waypoint.
        rationale:      Human-readable explanation of the bid.
    """
    agent_id: str = ""
    score: float = 0.0
    estimated_eta: float = 0.0
    battery_pct: float = 100.0
    distance: float = 0.0
    rationale: str = ""

    def __repr__(self) -> str:
        return (f"AgentBid({self.agent_id}, score={self.score:.2f}, "
                f"ETA={self.estimated_eta:.0f}s)")


# ── Agent Status ──────────────────────────────────────────────────────

@dataclass
class AgentStatus:
    """Heartbeat / status message broadcast by an agent."""
    agent_id: str = ""
    agent_type: str = ""
    state: AgentState = AgentState.IDLE
    position: Optional[GridPos] = None
    battery_pct: float = 100.0
    current_mission_id: Optional[str] = None
    capabilities: List[str] = field(default_factory=list)
    timestamp: float = field(default_factory=time.monotonic)


# ── Abstract Base Agent ──────────────────────────────────────────────

class HospitalAgent(ABC):
    """Abstract base class for all hospital agents.

    Lifecycle:
        1. Instantiate with a unique ID and capability set.
        2. Register with ``AgentRegistry`` (auto-publishes heartbeats).
        3. Receive task broadcasts via ``evaluate_mission()``.
        4. If selected, execute via ``accept_mission()``.
        5. Report completion or failure via EventBus.
    """

    def __init__(
        self,
        agent_type: str,
        capabilities: FrozenSet[str],
        home_position: GridPos = (5, 5),
    ) -> None:
        self.agent_id: str = f"{agent_type[:3].upper()}-{uuid.uuid4().hex[:6].upper()}"
```

```
        self.agent_type: str = agent_type
        self.capabilities: FrozenSet[str] = capabilities
        self.home_position: GridPos = home_position

        # State
        self._state: AgentState = AgentState.IDLE
        self._position: GridPos = home_position
        self._battery_pct: float = 100.0
        self._current_mission_id: Optional[str] = None
        self._last_heartbeat: float = time.monotonic()

        # Statistics
        self._missions_completed: int = 0
        self._missions_failed: int = 0
        self._total_distance: float = 0.0

        log.info("Agent %s (%s) created at %s", self.agent_id, agent_type,
                 home_position)

    # ── Properties ──────────────────────────────────────────────

    @property
    def state(self) -> AgentState:
        return self._state

    @property
    def position(self) -> GridPos:
        return self._position

    @property
    def battery_pct(self) -> float:
        return self._battery_pct

    @property
    def is_available(self) -> bool:
        """Agent can accept new tasks."""
        return self._state in (AgentState.IDLE, AgentState.RETURNING)

    @property
    def is_busy(self) -> bool:
        return self._state in (AgentState.EXECUTING, AgentState.ASSIGNED,
                               AgentState.BIDDING)

    # ── State Management ────────────────────────────────────────

    def set_state(self, new_state: AgentState) -> None:
        old = self._state
        self._state = new_state
        log.debug("Agent %s: %s → %s", self.agent_id, old.name, new_state.name)

    def set_position(self, pos: GridPos) -> None:
        self._position = pos

    def set_battery(self, pct: float) -> None:
        self._battery_pct = max(0.0, min(100.0, pct))

    # ── Heartbeat ───────────────────────────────────────────────

    def heartbeat(self) -> AgentStatus:
        """Generate and publish a heartbeat status message."""
        status = AgentStatus(
            agent_id=self.agent_id,
            agent_type=self.agent_type,
            state=self._state,
            position=self._position,
            battery_pct=self._battery_pct,
            current_mission_id=self._current_mission_id,
            capabilities=list(self.capabilities),
        )
        self._last_heartbeat = time.monotonic()
        EVENT_BUS.publish(
            HospitalEvent.AGENT_REGISTERED,
            data={"status": status},
            source=self.agent_id,
        )
        return status

    # ── Contract-Net Interface ──────────────────────────────────

    def has_capability(self, required: str) -> bool:
```

FILE: agents/delivery_agent.py [Lines 1–155 of 170]

```
"""
HospitalRobotNav — Delivery Agent
==================================
Specialised hospital agent for medicine, report, and sample delivery tasks.

Capabilities:
    - Medicine collection from pharmacy → delivery to patient room
    - Lab report transport (lab → doctor office)
    - Lab sample transport (ward → laboratory)
    - Wheelchair fetching

The agent bids aggressively on delivery-type missions and factors in
proximity to the pharmacy and current payload capacity.

Architecture Note (ROS2):
    ros2 run hospital_robot delivery_agent --ros-args -p agent_id:=DEL-001
"""

from __future__ import annotations

import math
from typing import Any, Dict, Optional, Tuple

from utilities.logger import get_logger
from .agent_base import AgentBid, AgentState, HospitalAgent
from . import agent_config as cfg

log = get_logger(__name__)

GridPos = Tuple[int, int]

# Delivery-related mission types this agent can handle
_DELIVERY_MISSIONS = frozenset({
    "MEDICINE_DELIVERY",
    "DELIVER_REPORT",
    "TRANSPORT_SAMPLE",
    "FETCH_WHEELCHAIR",
})


class DeliveryAgent(HospitalAgent):
    """Agent specialised in pick-up and delivery operations.

    Wraps a ``Robot`` entity and evaluates delivery missions based on
    proximity to the pharmacy / source location, battery level, and
    whether the agent is already carrying a payload.
    """

    def __init__(
        self,
        home_position: GridPos = (5, 5),
        pharmacy_position: GridPos = (50, 10),
    ) -> None:
        super().__init__(
            agent_type="delivery",
            capabilities=frozenset({
                cfg.CAPABILITY_DELIVER,
                cfg.CAPABILITY_TRANSPORT,
                cfg.CAPABILITY_FETCH,
            }),
            home_position=home_position,
        )
        self._pharmacy_pos = pharmacy_position
        self._carrying_item: Optional[str] = None
        self._carrying_destination: Optional[GridPos] = None

    # ── Properties ──────────────────────────────────────────────────

    @property
    def is_carrying(self) -> bool:
        return self._carrying_item is not None

    # ── Contract-Net ────────────────────────────────────────────────

    def evaluate_mission(
        self,
        mission_type: str,
        destination: Optional[GridPos],
        priority: int,
    ) -> Optional[AgentBid]:
```

```
        """Bid on delivery-type missions only."""
        if not self.is_available:
            return None

        if mission_type not in _DELIVERY_MISSIONS:
            return None  # Not our specialty

        if self._battery_pct < 15.0:
            log.debug("Agent %s declining — battery too low (%.0f%%)",
                      self.agent_id, self._battery_pct)
            return None

        # Compute distance to destination (or to pharmacy first)
        if destination:
            dist = self.grid_distance(destination)
        else:
            dist = self.grid_distance(self._pharmacy_pos)

        score = self.compute_bid_score(dist, priority)

        # Bonus: if already carrying an item for the same dest, bid higher
        if self.is_carrying and destination == self._carrying_destination:
            score = min(1.0, score + 0.15)

        # Penalty: if currently carrying something else
        if self.is_carrying and destination != self._carrying_destination:
            score *= 0.6

        eta = dist / max(cfg.DEFAULT_AGENT_SPEED, 0.1)

        bid = AgentBid(
            agent_id=self.agent_id,
            score=score,
            estimated_eta=eta,
            battery_pct=self._battery_pct,
            distance=dist,
            rationale=f"Delivery agent at dist={dist:.0f}, bat={self._battery_pct:.0f}%",
        )
        log.debug("Agent %s bid %.3f for %s", self.agent_id, score, mission_type)
        return bid

    def accept_mission(self, mission_id: str, mission_data: Dict[str, Any]) -> bool:
        """Accept a delivery mission and start execution."""
        if not self.is_available:
            log.warning("Agent %s cannot accept — state=%s",
                        self.agent_id, self._state.name)
            return False

        self._current_mission_id = mission_id
        self._carrying_item = mission_data.get("item")
        dest = mission_data.get("destination_pos")
        if dest:
            self._carrying_destination = tuple(dest)  # type: ignore[arg-type]
        self.set_state(AgentState.EXECUTING)

        log.info("Agent %s accepted mission %s — delivering '%s'",
                 self.agent_id, mission_id,
                 self._carrying_item or "item")
        return True

    def abort_mission(self, reason: str = "") -> None:
        """Abort current delivery and return to idle."""
        log.warning("Agent %s aborting mission %s: %s",
                    self.agent_id, self._current_mission_id, reason)
        self._carrying_item = None
        self._carrying_destination = None
        self._current_mission_id = None
        self.set_state(AgentState.IDLE)

    # ── Delivery lifecycle ──────────────────────────────────────

    def pickup_item(self, item_name: str, destination: GridPos) -> None:
        """Record that the agent picked up an item."""
        self._carrying_item = item_name
        self._carrying_destination = destination
```

FILE: agents/emergency_agent.py [Lines 1–140 of 149]

```
"""
HospitalRobotNav — Emergency Agent
====================================
High-priority agent that responds to emergency events.
```

```
Capabilities:
    - Immediate response to Code Blue / fire / urgent nurse calls
    - Can preempt other agents' missions
    - Always maintains minimum battery reserve for emergencies

Design Decision:
    The emergency agent bids with maximum score on priority-1 missions
    and declines all non-emergency tasks. This guarantees that the
    coordinator always has a dedicated responder.

Architecture Note (ROS2):
    ros2 run hospital_robot emergency_agent
    Subscribes to /hospital/emergency with QoS RELIABLE + TRANSIENT_LOCAL
    to never miss an alert.
"""

from __future__ import annotations

from typing import Any, Dict, Optional, Tuple

from utilities.event_bus import EVENT_BUS, HospitalEvent
from utilities.logger import get_logger
from .agent_base import AgentBid, AgentState, HospitalAgent
from . import agent_config as cfg

log = get_logger(__name__)

GridPos = Tuple[int, int]

_EMERGENCY_MISSIONS = frozenset({
    "EMERGENCY_RESPONSE",
    "CALL_NURSE",
})

# Minimum battery to remain on standby
_MIN_STANDBY_BATTERY = 25.0


class EmergencyAgent(HospitalAgent):
    """Agent that handles emergency events with highest priority.

    Always ready — only discharges below the standby reserve when
    actively responding to an emergency. Can preempt other agents
    by sending an ABORT via the EventBus.
    """

    def __init__(
        self,
        home_position: GridPos = (50, 50),
    ) -> None:
        super().__init__(
            agent_type="emergency",
            capabilities=frozenset({
                cfg.CAPABILITY_EMERGENCY,
            }),
            home_position=home_position,
        )
        self._emergency_active: bool = False
        self._emergency_location: Optional[GridPos] = None

        # Subscribe to emergency alerts for immediate response
        EVENT_BUS.subscribe(HospitalEvent.EMERGENCY_ALERT, self._on_emergency_alert)

    # ── Emergency listener ──────────────────────────────────────────

    def _on_emergency_alert(self, payload) -> None:
        """React to hospital emergency broadcasts."""
        room = payload.data.get("room", "unknown")
        code = payload.data.get("code", "RED")
        log.warning("Emergency agent %s received alert — room=%s, code=%s",
                    self.agent_id, room, code)
        self._emergency_active = True

    # ── Contract-Net ────────────────────────────────────────────────

    def evaluate_mission(
        self,
        mission_type: str,
        destination: Optional[GridPos],
        priority: int,
    ) -> Optional[AgentBid]:
        """Only bid on emergency / nurse-call missions."""
```

```
        if mission_type not in _EMERGENCY_MISSIONS:
            return None  # We only handle emergencies

        if self._state == AgentState.EXECUTING and not self._emergency_active:
            return None  # Already responding to another emergency

        if self._battery_pct < _MIN_STANDBY_BATTERY * 0.5:
            return None  # Critical battery — cannot respond

        dist = self.grid_distance(destination) if destination else 0.0

        # Emergency agents always bid high
        score = 0.95 if priority <= cfg.PRIORITY_HIGH else 0.80
        eta = dist / max(cfg.DEFAULT_AGENT_SPEED * 1.5, 0.1)  # Rush speed

        return AgentBid(
            agent_id=self.agent_id,
            score=score,
            estimated_eta=eta,
            battery_pct=self._battery_pct,
            distance=dist,
            rationale=f"Emergency agent — priority response, dist={dist:.0f}",
        )

    def accept_mission(self, mission_id: str, mission_data: Dict[str, Any]) -> bool:
        """Accept and immediately start emergency response."""
        self._current_mission_id = mission_id
        self._emergency_location = mission_data.get("destination_pos")
        self._emergency_active = True
        self.set_state(AgentState.EXECUTING)

        log.info("EMERGENCY Agent %s responding to mission %s",
                 self.agent_id, mission_id)
        return True

    def abort_mission(self, reason: str = "") -> None:
        """Abort emergency response (only if another agent takes over)."""
        log.warning("Emergency agent %s aborting mission %s: %s",
                    self.agent_id, self._current_mission_id, reason)
        self._emergency_active = False
        self._emergency_location = None
        self._current_mission_id = None
        self.set_state(AgentState.IDLE)

    # ── Emergency lifecycle ──────────────────────────────────────────

    def resolve_emergency(self) -> None:
        """Mark the current emergency as resolved."""
        self._emergency_active = False
        self._emergency_location = None
        log.info("Emergency agent %s — emergency resolved", self.agent_id)

    @property
```

MODULE 3: MISSION MANAGEMENT

FILE: missions/mission_types.py [Lines 25–160 of 237]

```
# ──────────────────────────────────────────────────────────────────────
# Task Types
# ──────────────────────────────────────────────────────────────────────

class TaskType(Enum):
    """Atomic task categories for mission steps."""
    NAVIGATE        = auto()   # Move to a grid position
    COLLECT         = auto()   # Pick up an item at current location
    DELIVER         = auto()   # Drop off an item at current location
    WAIT            = auto()   # Wait at current location for a duration
    INTERACT        = auto()   # Interact with a person (speak, alert)
    ESCORT          = auto()   # Lead a person to a destination
    ALERT           = auto()   # Trigger an alert (nurse call, emergency)
    RETURN_TO_DOCK  = auto()   # Navigate back to charging dock
    CHARGE          = auto()   # Charge at dock


class TaskStatus(Enum):
    """Execution status of a single task step."""
    PENDING     = auto()
    IN_PROGRESS = auto()
    COMPLETED   = auto()
    FAILED      = auto()
    SKIPPED     = auto()


# ──────────────────────────────────────────────────────────────────────
# Task Step
# ──────────────────────────────────────────────────────────────────────

@dataclass
class TaskStep:
    """A single executable step in a mission plan.

    Attributes:
        step_id:      Unique step identifier.
        task_type:    What kind of action to perform.
        description:  Human-readable description.
        destination:  Target location name (for NAVIGATE steps).
        destination_pos: Resolved grid coordinates (row, col).
        item:         Item to collect/deliver (for COLLECT/DELIVER).
        duration_sec: Expected duration in seconds (for WAIT steps).
        status:       Current execution status.
        started_at:   Timestamp when step execution began.
        completed_at: Timestamp when step completed.
        metadata:     Arbitrary extra data for specialised steps.
    """
    step_id: str = ""
    task_type: TaskType = TaskType.NAVIGATE
    description: str = ""
    destination: str = ""
    destination_pos: Optional[GridPos] = None
    item: str = ""
    duration_sec: float = 0.0
    status: TaskStatus = TaskStatus.PENDING
    started_at: float = 0.0
    completed_at: float = 0.0
    metadata: Dict[str, Any] = field(default_factory=dict)

    def __post_init__(self) -> None:
        if not self.step_id:
            self.step_id = str(uuid.uuid4())[:8].upper()

    def start(self) -> None:
        """Mark step as in progress."""
        self.status = TaskStatus.IN_PROGRESS
        self.started_at = time.monotonic()

    def complete(self) -> None:
        """Mark step as completed."""
        self.status = TaskStatus.COMPLETED
        self.completed_at = time.monotonic()

    def fail(self) -> None:
        """Mark step as failed."""
```

```
        self.status = TaskStatus.FAILED
        self.completed_at = time.monotonic()

    @property
    def elapsed_sec(self) -> float:
        """Seconds elapsed since step started (0 if not started)."""
        if self.started_at == 0.0:
            return 0.0
        end = self.completed_at if self.completed_at > 0 else time.monotonic()
        return end - self.started_at

    @property
    def is_complete(self) -> bool:
        return self.status == TaskStatus.COMPLETED

    @property
    def is_active(self) -> bool:
        return self.status == TaskStatus.IN_PROGRESS

    @property
    def is_pending(self) -> bool:
        return self.status == TaskStatus.PENDING

    def __repr__(self) -> str:
        return (
            f"TaskStep({self.step_id}, {self.task_type.name}, "
            f"'{self.description[:30]}', {self.status.name})"
        )


# ─────────────────────────────────────────────────────────────────────────────
# Mission Plan
# ─────────────────────────────────────────────────────────────────────────────

@dataclass
class MissionPlan:
    """An ordered sequence of TaskSteps forming a complete mission.

    Attributes:
        plan_id:      Unique plan identifier.
        mission_type: High-level mission category string.
        description:  Human-readable mission description.
        steps:        Ordered list of TaskSteps.
        priority:     Mission priority (1=critical, 5=low).
        created_at:   Timestamp when plan was generated.
        eta_total_sec: Estimated total execution time in seconds.
        raw_command:  The original spoken command.
    """
    plan_id: str = ""
    mission_type: str = ""
    description: str = ""
    steps: List[TaskStep] = field(default_factory=list)
    priority: int = 4
    created_at: float = field(default_factory=time.monotonic)
    eta_total_sec: float = 0.0
    raw_command: str = ""

    def __post_init__(self) -> None:
        if not self.plan_id:
            self.plan_id = f"MSN-{str(uuid.uuid4())[:6].upper()}"
```

FILE: missions/medication_workflow.py [Lines 25–170 of 303]

```
from utilities.event_bus import EVENT_BUS, HospitalEvent
from utilities.logger import get_logger

log = get_logger(__name__)


class MedicationWorkflowState(Enum):
    """Lifecycle stages for a clinical medication delivery order."""
    SCHEDULED          = auto()   # In database queue, awaiting due time or trigger
    DISPATCHED         = auto()   # Assigned to robot, route planned to pharmacy
    EN_ROUTE_PICKUP    = auto()   # Moving to Pharmacy
    COLLECTING         = auto()   # Dispensing/loading medicine at pharmacy
    EN_ROUTE_PATIENT   = auto()   # Transporting medicine to patient ward/room
    BEDSIDE_VERIFYING  = auto()   # Arrived at patient bedside; barcode/identity check
    DELIVERED          = auto()   # Successfully handed off; DB updated
    CANCELLED          = auto()   # Aborted before completion
    FAILED             = auto()   # Encountered unrecoverable error (e.g. mismatch)
```

```python
@dataclass
class MedicationDeliveryOrder:
    """Represents a single active or historical medication delivery mission."""
    order_id: str = field(default_factory=lambda: f"ORD-{uuid.uuid4().hex[:6].upper()}")
    schedule_id: str = ""
    patient_id: str = ""
    patient_name: str = ""
    room_id: str = ""
    medicine_name: str = ""
    dosage: str = ""
    route: str = "oral"
    assigned_robot_id: str = "arogya"
    state: MedicationWorkflowState = MedicationWorkflowState.SCHEDULED
    verification_passed: bool = False
    verification_notes: str = ""
    dispatched_at: float = 0.0
    collected_at: float = 0.0
    delivered_at: float = 0.0
    db_task_id: Optional[str] = None

    @property
    def is_active(self) -> bool:
        return self.state not in (
            MedicationWorkflowState.DELIVERED,
            MedicationWorkflowState.CANCELLED,
            MedicationWorkflowState.FAILED,
        )

    def to_dict(self) -> Dict[str, Any]:
        return {
            "order_id": self.order_id,
            "schedule_id": self.schedule_id,
            "patient_id": self.patient_id,
            "patient_name": self.patient_name,
            "room_id": self.room_id,
            "medicine_name": self.medicine_name,
            "dosage": self.dosage,
            "route": self.route,
            "assigned_robot_id": self.assigned_robot_id,
            "state": self.state.name,
            "verification_passed": self.verification_passed,
            "verification_notes": self.verification_notes,
            "dispatched_at": self.dispatched_at,
            "collected_at": self.collected_at,
            "delivered_at": self.delivered_at,
        }


class MedicationWorkflowManager:
    """Coordinates end-to-end medication deliveries between Database, Robots, and Events."""

    def __init__(
        self,
        db: HospitalDatabase,
        query_engine: DBQueryEngine,
    ) -> None:
        self._db = db
        self._query_engine = query_engine
        self._orders: Dict[str, MedicationDeliveryOrder] = {}

        log.info("MedicationWorkflowManager initialised.")

    def create_order_for_patient(
        self,
        patient_id: str,
        medicine_name: str,
        assigned_robot: str = "arogya",
        priority: int = 2,
    ) -> Optional[MedicationDeliveryOrder]:
        """Create and dispatch a medication delivery order directly from a patient ID."""
        pid = self._query_engine.resolve_patient_id(patient_id) or patient_id
        patient_info = self._query_engine.get_patient_info(pid)
        if not patient_info:
            log.warning("MedicationWorkflowManager: Cannot create order — patient %s not found.", pid)
            return None

        # Check stock in database
        med_row = self._db.get_medicine_by_name(medicine_name)
        if not med_row:
            log.warning("MedicationWorkflowManager: Medicine '%s' not in formulary.", medicine_name)
            return None

        room = patient_info.get("assigned_room") or patient_info.get("room_id") or "Ward 101"
```

```
        order = MedicationDeliveryOrder(
            patient_id=pid,
            patient_name=patient_info.get("name", "Unknown"),
            room_id=room,
            medicine_name=med_row["name"],
            dosage=med_row.get("dosage", "Standard"),
            assigned_robot_id=assigned_robot,
            state=MedicationWorkflowState.DISPATCHED,
            dispatched_at=time.time(),
        )

        # Record in hospital_tasks table
        task_res = self._query_engine.create_medication_delivery_task(
            patient_id=pid,
            medicine_name=med_row["name"],
        )
        task_id = task_res.get("task_id") if isinstance(task_res, dict) else str(task_res)
        order.db_task_id = task_id
        self._orders[order.order_id] = order

        EVENT_BUS.publish(
            HospitalEvent.MISSION_STARTED,
            data={"order": order.to_dict()},
            source="medication_workflow",
        )
        log.info("Medication delivery order created: %s for %s (%s)", order.order_id, pid, medicine_name)
        return order

    def dispatch_from_schedule(
        self,
        schedule_id: str,
        assigned_robot: str = "arogya",
    ) -> Optional[MedicationDeliveryOrder]:
        """Dispatch a delivery order from an existing medication_schedule row."""
        with self._db._cursor() as c:
            c.execute("""
                SELECT ms.*, p.name as patient_name, p.room_id,
                       rx.medicine_id, rx.dosage, rx.route,
                       m.name as medicine_name
                FROM medication_schedules ms
                JOIN patients p ON ms.patient_id = p.patient_id
                JOIN prescriptions rx ON ms.prescription_id = rx.prescription_id
                JOIN medicines m ON rx.medicine_id = m.medicine_id
                WHERE ms.schedule_id = ?
```

MODULE 4: NVIDIA ISAAC SIM INTEGRATION

FILE: isaac_sim_hospital/configs/simulation_config.py [Lines 1–140 of 184]

```
"""
Isaac Sim Hospital Robot — Simulation Configuration
====================================================
VRAM-optimized settings for RTX 4050 Laptop GPU (6 GB).

All simulation scripts import these settings to ensure consistent,
stable performance on the target hardware.

Usage:
    from configs.simulation_config import SIM_CONFIG, get_app_config
    app = SimulationApp(get_app_config())
"""

from __future__ import annotations


# ─────────────────────────────────────────────────────────────────────────────
# Quality Profiles (Phase 2 Performance Optimization)
# ─────────────────────────────────────────────────────────────────────────────

QUALITY_PROFILES = {
    "performance": {
        "description": "High-throughput profile for resource-constrained systems (maximum FPS)",
        "width": 1024,
        "height": 576,
        "window_width": 1280,
        "window_height": 720,
        "anti_aliasing": 0,
        "renderer": "RayTracedLighting",
        "max_bounces": 0,
        "max_specular_transmission_bounces": 0,
        "denoiser": False,
        "subdiv_refinement_level": 0,
        "sensor_update_hz": 8.0,
        "dashboard_update_hz": 4.0,
        "ros2_publish_hz": 8.0,
    },
    "balanced": {
        "description": "Balanced default profile for RTX 4050 6GB (fluid 30-60 FPS, crisp visuals)",
        "width": 1280,
        "height": 720,
        "window_width": 1440,
        "window_height": 900,
        "anti_aliasing": 0,
        "renderer": "RayTracedLighting",
        "max_bounces": 1,
        "max_specular_transmission_bounces": 0,
        "denoiser": True,
        "subdiv_refinement_level": 0,
        "sensor_update_hz": 10.0,
        "dashboard_update_hz": 8.0,
        "ros2_publish_hz": 10.0,
    },
    "quality": {
        "description": "Fidelity-focused profile for high-end GPUs / presentation recordings",
        "width": 1600,
        "height": 900,
        "window_width": 1680,
        "window_height": 1050,
        "anti_aliasing": 1,
        "renderer": "RayTracedLighting",
        "max_bounces": 2,
        "max_specular_transmission_bounces": 1,
        "denoiser": True,
        "subdiv_refinement_level": 1,
        "sensor_update_hz": 15.0,
        "dashboard_update_hz": 12.0,
        "ros2_publish_hz": 15.0,
    },
}

ACTIVE_PROFILE_NAME = "balanced"


def get_app_config(headless: bool = False, profile: str = "balanced") -> dict:
```

```
    """Return SimulationApp configuration optimized for RTX 4050 Laptop.

    Args:
        headless: If True, run without GUI (for automated testing).
        profile: 'performance', 'balanced' (default), or 'quality'.

    Returns:
        Configuration dict for SimulationApp().
    """
    global ACTIVE_PROFILE_NAME
    p_name = profile.lower() if profile.lower() in QUALITY_PROFILES else "balanced"
    ACTIVE_PROFILE_NAME = p_name
    prof = QUALITY_PROFILES[p_name]

    # Dynamically apply frequency throttles to AROGYA_FEATURES
    AROGYA_FEATURES["sensor_update_hz"] = prof.get("sensor_update_hz", 10.0)
    AROGYA_FEATURES["dashboard_update_hz"] = prof.get("dashboard_update_hz", 8.0)
    AROGYA_FEATURES["ros2_publish_hz"] = prof.get("ros2_publish_hz", 10.0)

    return {
        "headless": headless,
        "hide_ui": False,
        "width": prof["width"],
        "height": prof["height"],
        "window_width": prof["window_width"],
        "window_height": prof["window_height"],
        "multi_gpu": False,
        "active_gpu": 0,
        "physics_gpu": 0,
        "extra_args": [
            "--/renderer/multiGpu/enabled=false",
            "--/app/fastShutdown=true",
        ],
        "anti_aliasing": prof["anti_aliasing"],
        "renderer": prof["renderer"],
        "max_bounces": prof["max_bounces"],
        "max_specular_transmission_bounces": prof["max_specular_transmission_bounces"],
        "denoiser": prof["denoiser"],
        "subdiv_refinement_level": prof["subdiv_refinement_level"],
    }


# ─────────────────────────────────────────────────────────────────────────────
# Simulation Parameters
# ─────────────────────────────────────────────────────────────────────────────

SIM_CONFIG = {
    # Physics
    "physics_dt": 1.0 / 60.0,        # 60 Hz physics
    "rendering_dt": 1.0 / 30.0,      # 30 Hz rendering (saves GPU)
    "stage_units_in_meters": 1.0,     # 1 USD unit = 1 meter

    # Hospital environment scale
    "hospital_width_m": 20.0,         # 20 meters wide
    "hospital_depth_m": 15.0,         # 15 meters deep
    "wall_height_m": 2.8,             # 2.8 meter walls
    "wall_thickness_m": 0.15,         # 15 cm thick walls
    "door_width_m": 1.2,              # 1.2 meter doorways
    "corridor_width_m": 2.5,          # 2.5 meter corridors

    # Lighting (balanced, non-glare)
    "distant_light_intensity": 380,
    "point_light_intensity": 160,
    "point_light_radius": 5.0,
```

FILE: isaac_sim_hospital/robots/robot_state_machine.py [Lines 50–185 of 203]

```
# ─────────────────────────────────────────────────────────────────────────────
# Robot State Enum
# ─────────────────────────────────────────────────────────────────────────────

class RobotState(Enum):
    """Operational states of the Arogya hospital AMR."""
    IDLE                  = auto()
    AVAILABLE             = IDLE
    ASSIGNED              = auto()
    TASK_ASSIGNED         = ASSIGNED
    MISSION_ANALYSIS      = auto()
    PLANNING              = auto()
    NAVIGATING            = auto()
    STUCK_DETECTED        = auto()
    LOCAL_RECOVERY        = auto()
```

```
    OBSTACLE_DETECTED      = auto()
    AVOIDING_OBSTACLE      = auto()
    AVOIDING               = AVOIDING_OBSTACLE
    REPLANNING             = auto()
    WAITING_FOR_CLEARANCE  = auto()
    EXECUTING_TASK         = auto()
    MISSION_COMPLETE       = auto()
    NAVIGATION_FAILED      = auto()
    BLOCKED                = auto()
    MISSION_BLOCKED        = BLOCKED
    EMERGENCY_OVERRIDE     = auto()
    RETURNING              = auto()
    RETURNING_TO_DOCK      = RETURNING
    DOCKING                = auto()
    CHARGING               = auto()
    PAUSED                 = auto()


# ──────────────────────────────────────────────────────────────────────
# Priority Hierarchy Definition
# ──────────────────────────────────────────────────────────────────────

PRIORITY_LEVELS: Dict[int, Dict[str, Any]] = {
    1: {"name": "CRITICAL_EMERGENCY", "speed_multiplier": 1.25, "preemptable": False},
    2: {"name": "URGENT_MEDICINE",    "speed_multiplier": 1.10, "preemptable": True},
    3: {"name": "SCHEDULED_DELIVERY", "speed_multiplier": 1.00, "preemptable": True},
    4: {"name": "VISITOR_ESCORT",     "speed_multiplier": 0.85, "preemptable": True},
    5: {"name": "RETURN_TO_DOCK",     "speed_multiplier": 0.80, "preemptable": True},
}


# ──────────────────────────────────────────────────────────────────────
# Robot State Machine
# ──────────────────────────────────────────────────────────────────────

class RobotStateMachine:
    """
    Finite State Machine managing AMR lifecycle and operational transitions.
    """

    def __init__(self, initial_state: RobotState = RobotState.IDLE) -> None:
        self.current_state: RobotState = initial_state
        self.previous_state: Optional[RobotState] = None
        self.current_reason: str = "Initialization"
        self.current_action: str = "Standby"
        self.state_enter_time: float = time.monotonic()
        self.history: List[Tuple[float, RobotState, str]] = [
            (self.state_enter_time, initial_state, "Initialization")
        ]
        self._on_transition_callbacks: List[Callable[[RobotState, RobotState, str], None]] = []

    def register_transition_callback(
        self, callback: Callable[[RobotState, RobotState, str], None]
    ) -> None:
        """Register a callback function to be invoked on every state change."""
        self._on_transition_callbacks.append(callback)

    def transition_to(self, new_state: RobotState, reason: str = "", action: str = "") -> bool:
        """
        Transition robot to a new operational state with explicit reason and action.

        Args:
            new_state: Target RobotState.
            reason: Contextual description of the transition trigger.
            action: Concrete operational action being taken (e.g. 'Replanning', 'Tracking').

        Returns:
            True if transition occurred, False if already in that state.
        """
        if new_state == self.current_state:
            if reason:
                self.current_reason = reason
            if action:
                self.current_action = action
            return False

        now = time.monotonic()
        prev = self.current_state
        self.previous_state = prev
        self.current_state = new_state
        self.current_reason = reason or "Normal operation"
        self.current_action = action or ("Navigating" if new_state == RobotState.NAVIGATING else "Standby")
```

```
        self.state_enter_time = now

        log_msg = f"[FSM] {prev.name} -> {new_state.name} | Reason: {self.current_reason} | Action:
            {self.current_action}"
        if new_state == RobotState.EMERGENCY_OVERRIDE:
            log.warning(log_msg)
        else:
            log.info(log_msg)

        self.history.append((now, new_state, reason))

        # Notify callbacks (e.g. voice engine, UI HUD)
        for cb in self._on_transition_callbacks:
            try:
                cb(prev, new_state, reason)
            except Exception as e:
                log.error("[FSM] Error in transition callback: %s", e)

        return True

    @property
    def state(self) -> RobotState:
        """Return the current RobotState."""
        return self.current_state

    @property
    def state_name(self) -> str:
        """Return human-readable state string."""
        return self.current_state.name

    def time_in_state(self) -> float:
        """Return elapsed duration in seconds since current state entry."""
        return time.monotonic() - self.state_enter_time

    def is_idle(self) -> bool:
        return self.current_state == RobotState.IDLE

    def is_navigating(self) -> bool:
        return self.current_state in (RobotState.NAVIGATING, RobotState.EMERGENCY_OVERRIDE)
```

MODULE 5: ROS2 AND ROBOT CONTROL

FILE: hospital_robot_nav_ws/src/hospital_robot_nav_nodes/hospital_robot_nav_nodes/robot_controller_node.py [Lines 35–200 of 324]

```
class RobotControllerNode(Node):
    """Publishes robot odometry/state, TF transforms, and follows /plan trajectories."""

    def __init__(self) -> None:
        super().__init__("robot_controller_node")

        # ── Parameters ──────────────────────────────────────────────
        self.declare_parameter("publish_rate_hz", 50.0)
        self.declare_parameter("start_row", 80)
        self.declare_parameter("start_col", 11)

        rate = self.get_parameter("publish_rate_hz").value
        start = (
            self.get_parameter("start_row").value,
            self.get_parameter("start_col").value,
        )

        # ── Core robot + physics ────────────────────────────────────
        self._robot = Robot(start_pos=start)
        self._physics = RobotPhysics()

        # ── TF Broadcaster ──────────────────────────────────────────
        self._tf_broadcaster = TransformBroadcaster(self)

        # ── Subscribers / Publishers ────────────────────────────────
        self._sub_cmd_vel = self.create_subscription(
            Twist, "/robot/cmd_vel", self._on_cmd_vel, 10
        )
        self._sub_plan = self.create_subscription(
            Path, "/plan", self._on_plan_path, 10
        )
        self._pub_odom = self.create_publisher(Odometry, "/odom", 10)
        self._pub_state = self.create_publisher(String, "/robot/state", 10)
        self._pub_battery = self.create_publisher(BatteryState, "/battery", 10)
        self._pub_robot_marker = self.create_publisher(Marker, "/robot/marker", 10)
        self._pub_robot_markers = self.create_publisher(MarkerArray, "/robot/markers", 10)

        # ── Timer ───────────────────────────────────────────────────
        self._timer = self.create_timer(1.0 / max(rate, 1.0), self._publish_state_all)
        self._last_step = self.get_clock().now()

        self.get_logger().info("RobotControllerNode initialised with path-following.")

    # ── Path & Velocity Callbacks ───────────────────────────────────

    def _on_cmd_vel(self, msg: Twist) -> None:
        """Set the robot's target velocity from a Twist command."""
        self._robot.target_velocity = max(0.0, msg.linear.x)

    def _on_plan_path(self, msg: Path) -> None:
        """Receive a planned trajectory from RViz / planner and start navigating."""
        if not msg.poses:
            return
        waypoints = []
        for ps in msg.poses:
            r = int(round(ps.pose.position.y / 0.5))
            c = int(round(ps.pose.position.x / 0.5))
            waypoints.append((r, c))

        if waypoints:
            self._robot.set_path(waypoints)
            self._robot.target_velocity = 1.4  # Target speed in grid cells/s (~0.7 m/s)
            self._robot.transition(RobotState.MOVING)
            self.get_logger().info(f"Received new path with {len(waypoints)} waypoints. Robot moving!")

    # ── Periodic state publish & waypoint navigation ────────────────

    def _publish_state_all(self) -> None:
        now = self.get_clock().now()
        dt = (now - self._last_step).nanoseconds / 1e9
        self._last_step = now
        dt = min(max(dt, 1e-4), 0.1)

        # ── Follow waypoints if path is active ──────────────────────
```

```
        if self._robot.has_path():
            wp = self._robot.get_next_waypoint()
            if wp is not None:
                dr = wp[0] - self._robot.row
                dc = wp[1] - self._robot.col
                dist = math.hypot(dr, dc)

                if dist < 0.4:
                    self._robot.advance_waypoint()
                    if not self._robot.has_path():
                        # Goal reached
                        self._robot.target_velocity = 0.0
                        self._robot.velocity = 0.0
                        self._robot.transition(RobotState.IDLE)
                        self.get_logger().info("Robot reached destination successfully!")
                else:
                    # Steer heading towards waypoint
                    target_hdg = math.atan2(dc, dr)
                    diff = (target_hdg - self._robot.heading + math.pi) % (2 * math.pi) - math.pi
                    self._robot.heading += diff * min(1.0, dt * 5.0)
                    self._robot.target_velocity = 1.4

        # Advance the robot physics
        if self._robot.velocity > 0.0 or self._robot.target_velocity > 0.0:
            self._physics.step(self._robot, dt)

        # ── Odometry ──────────────────────────────────────────────
        odom = Odometry()
        odom.header.stamp = now.to_msg()
        odom.header.frame_id = "map"
        odom.child_frame_id = "base_footprint"

        # Grid (row, col) → ROS (x, y): col→x, row→y (inverted direction)
        odom.pose.pose.position.x = float(self._robot.col) * 0.5
        odom.pose.pose.position.y = float(self._robot.row) * 0.5
        odom.pose.pose.orientation.z = math.sin(self._robot.heading / 2.0)
        odom.pose.pose.orientation.w = math.cos(self._robot.heading / 2.0)

        odom.twist.twist.linear.x = float(self._robot.velocity) * 0.5
        odom.twist.twist.angular.z = float(self._robot.angular_vel)
        self._pub_odom.publish(odom)

        # ── TF Transform: map -> base_footprint ──────────────────
        tf_msg = TransformStamped()
        tf_msg.header.stamp = now.to_msg()
        tf_msg.header.frame_id = "map"
        tf_msg.child_frame_id = "base_footprint"
        tf_msg.transform.translation.x = float(self._robot.col) * 0.5
        tf_msg.transform.translation.y = float(self._robot.row) * 0.5
        tf_msg.transform.translation.z = 0.0
        tf_msg.transform.rotation.z = math.sin(self._robot.heading / 2.0)
        tf_msg.transform.rotation.w = math.cos(self._robot.heading / 2.0)
        self._tf_broadcaster.sendTransform(tf_msg)

        # ── 3D Humanoid Robot Multi-Part Markers ──────────────────
        m_stamp = now.to_msg()
        m_array = MarkerArray()

        # 1. Base Mobile Chassis (Dark Navy Cylinder)
        m_base = Marker()
        m_base.header.stamp = m_stamp
        m_base.header.frame_id = "base_link"
        m_base.ns = "robot_parts"
        m_base.id = 0
        m_base.type = Marker.CYLINDER
        m_base.action = Marker.ADD
        m_base.pose.position.z = 0.15
        m_base.pose.orientation.w = 1.0
        m_base.scale.x = 0.55
        m_base.scale.y = 0.55
        m_base.scale.z = 0.3
        m_base.color.r = 0.08
        m_base.color.g = 0.12
        m_base.color.b = 0.22
        m_base.color.a = 0.98
        m_array.markers.append(m_base)

        # 2. Sleek Humanoid Torso (Medical White / Cyan)
        m_torso = Marker()
        m_torso.header.stamp = m_stamp
        m_torso.header.frame_id = "base_link"
        m_torso.ns = "robot_parts"
        m_torso.id = 1
```

```
        m_torso.type = Marker.CYLINDER
        m_torso.action = Marker.ADD
        m_torso.pose.position.z = 0.65
        m_torso.pose.orientation.w = 1.0
        m_torso.scale.x = 0.42
        m_torso.scale.y = 0.35
        m_torso.scale.z = 0.70
        m_torso.color.r = 0.95
        m_torso.color.g = 0.97
```

FILE: controllers/kinematics.py [Lines 1–145 of 152]

```
"""
HospitalRobotNav — Robot Physics (Kinematics)
=============================================
Simulates differential drive robot kinematics for smooth animation.

Physics model:
    Linear velocity:  Controlled by PD controller toward target speed
    Angular velocity: Proportional to heading error
    Position update:  Euler integration at dt
"""

from __future__ import annotations

import math
from typing import Tuple

from config.robot_config import PHYSICS_CONFIG
from robot.robot import Robot
from utilities.geometry import angle_difference, normalize_angle


class RobotPhysics:
    """
    Updates robot position and heading each simulation frame.

    Uses a simplified differential-drive kinematic model:
        x(t+dt) = x(t) + v(t) * cos(θ) * dt
        y(t+dt) = y(t) - v(t) * sin(θ) * dt   (row increases downward)
        θ(t+dt) = θ(t) + ω(t) * dt

    Where ω is controlled to minimize heading error to the next waypoint.
    """

    def __init__(self) -> None:
        # PD gains for speed controller
        self._kp_speed:   float = 3.0
        self._kp_heading: float = 4.0
        # Derivative (velocity-damping) gain for the heading controller.
        # Damping proportional to the *current* angular velocity prevents
        # the robot from swinging past the target heading and circling
        # endlessly (the pure-P controller was marginally stable at best).
        self._kd_heading: float = 1.0

    def step(self, robot: Robot, dt: float) -> None:
        """
        Advance robot physics by one time step.

        Moves robot toward its next waypoint using smooth kinematic control.

        Args:
            robot: Robot instance (mutated in place).
            dt:    Time step in seconds.
        """
        waypoint = robot.get_next_waypoint()
        if waypoint is None:
            # No waypoint — decelerate to stop
            self._decelerate(robot, dt)
            return

        wr, wc = float(waypoint[0]), float(waypoint[1])
        dr = wr - robot.row
        dc = wc - robot.col
        dist = math.hypot(dr, dc)

# ── Check arrival at waypoint ──────────────────────────────
        # Use a small, fixed tolerance so smoothed sub-cell waypoints are
        # actually traversed.  The old value (RADIUS_CELLS * 0.8 = 0.56)
        # was larger than the distance between consecutive Bezier sub-cell
        # waypoints (~0.1 cells), so the robot "arrived" at the first one
        # instantly and never made forward progress.
```

```
        arrival_tol = 0.25
        if dist < arrival_tol:
            # Skip waypoints that are too close — don't return immediately,
            # keep moving through the path. Advance past any waypoints that
            # are within the tolerance distance of the current position.
            while dist < arrival_tol:
                has_more = robot.advance_waypoint()
                if not has_more:
                    from robot.robot_state import RobotState
                    robot.transition(RobotState.ARRIVED)
                    return
                # Re-check distance to the *next* waypoint
                waypoint = robot.get_next_waypoint()
                if waypoint is None:
                    from robot.robot_state import RobotState
                    robot.transition(RobotState.ARRIVED)
                    return
                wr, wc = float(waypoint[0]), float(waypoint[1])
                dr = wr - robot.row
                dc = wc - robot.col
                dist = math.hypot(dr, dc)
            # Found a waypoint beyond tolerance — keep moving toward it
            # (fall through to the movement code below)

        # ── Desired heading ──────────────────────────────────────────
        desired_heading = math.atan2(-dr, dc)   # atan2(-Δrow, Δcol)

        # Smooth the target heading by intentionally ignoring tiny angular
        # differences. This prevents the robot from "fighting" its own motion
        # when the path contains close, sub-cell waypoint jitter from smoothing.
        heading_error = angle_difference(desired_heading, robot.heading)
        if abs(heading_error) < math.radians(2.5):
            heading_error = 0.0
            robot.angular_vel = 0.0
            target_speed = PHYSICS_CONFIG.CRUISE_SPEED
        else:
            # PD controller for angular velocity with a deadband to reduce
            # unnecessary twirling around the target heading.
            robot.angular_vel = (self._kp_heading * heading_error
                                 - self._kd_heading * robot.angular_vel)
            robot.angular_vel = max(
                -PHYSICS_CONFIG.MAX_ANGULAR_VELOCITY,
                min(PHYSICS_CONFIG.MAX_ANGULAR_VELOCITY, robot.angular_vel),
            )

            # ── Linear velocity control ──────────────────────────────
            # Slow down (and stop if heading error is large) so the robot
            # turns toward the target before driving forward. Driving while
            # facing away from the target makes the robot circle endlessly.
            if abs(heading_error) > math.radians(45):
                target_speed = 0.0
            else:
                heading_factor = max(0.35, 1.0 - abs(heading_error) / math.radians(90))
                target_speed = PHYSICS_CONFIG.CRUISE_SPEED * heading_factor

        # Use the tuned target speed even when the heading is well-aligned and
        # the robot is still moving smoothly, so it doesn't alternately brake and
        # accelerate around small path jitter.

        speed_error   = target_speed - robot.velocity
        robot.velocity += self._kp_speed * speed_error * dt
        robot.velocity  = max(
            0.0,
            min(PHYSICS_CONFIG.MAX_LINEAR_VELOCITY, robot.velocity),
        )

        # ── Integrate heading ────────────────────────────────────────
        robot.heading = normalize_angle(robot.heading + robot.angular_vel * dt)

        # ── Integrate position ───────────────────────────────────────
        new_row = robot.row + robot.velocity * (-math.sin(robot.heading)) * dt
        new_col = robot.col + robot.velocity * math.cos(robot.heading) * dt

        robot.set_position(new_row, new_col)
        robot.time_moving += dt
```

MODULE 6: NAVIGATION AND PATH PLANNING

FILE: algorithms/hybrid_astar.py [Lines 55–220 of 342]

```
# ─────────────────────────────────────────────────────────────────────────
# Continuous State
# ─────────────────────────────────────────────────────────────────────────

@dataclass(frozen=True)
class HybridState:
    """
    Continuous configuration state for Hybrid A*.

    Attributes:
        x:      Continuous column position (cells).
        y:      Continuous row position (cells).
        theta:  Heading angle (radians), discretized for hashing.
    """
    x:     float
    y:     float
    theta: float

    @property
    def grid_pos(self) -> GridPos:
        """Snap continuous position to nearest grid cell (row, col)."""
        return (int(round(self.y)), int(round(self.x)))

    def discretize(self, xy_res: float = 1.0, theta_res: float = 0.5) -> Tuple[int, int, int]:
        """
        Convert to discrete key for hash map.

        Args:
            xy_res:    Resolution for x/y (cells).
            theta_res: Resolution for theta (radians).

        Returns:
            (xi, yi, thetai) integer key.
        """
        xi     = int(round(self.x / xy_res))
        yi     = int(round(self.y / xy_res))
        # Wrap theta to [0, 2π) then discretize
        theta_wrapped = self.theta % (2 * math.pi)
        thetai = int(round(theta_wrapped / theta_res)) % int(round(2 * math.pi / theta_res))
        return (xi, yi, thetai)


# ─────────────────────────────────────────────────────────────────────────
# Motion Primitives
# ─────────────────────────────────────────────────────────────────────────

def _generate_motion_primitives(
    step_size: float = 1.0,
    n_angles: int = 7,
    max_steer: float = math.pi / 2,
) -> List[Tuple[float, float, float, float]]:
    """
    Generate kinematically feasible motion primitives for differential drive robot.

    Each primitive: (step, dtheta, cost, steer)
    Includes forward motions, in-place rotations, and reverse motions.
    """
    primitives = []
    # Forward motions with steering angles up to ±pi/2
    steer_angles = [
        -max_steer + i * (2 * max_steer / (n_angles - 1))
        for i in range(n_angles)
    ]
    for steer in steer_angles:
        dtheta = steer
        cost = step_size * (1.0 + 0.3 * abs(steer) / max_steer)
        primitives.append((step_size, dtheta, cost, steer))

    # In-place rotations for differential drive
    for rot in (-math.pi / 2, -math.pi / 4, math.pi / 4, math.pi / 2):
        primitives.append((0.0, rot, 0.4, rot))

    # Reverse motions for tight navigation
    for steer in (0.0, -math.pi / 4, math.pi / 4):
        primitives.append((-step_size, steer, step_size * 1.5, steer))
```

```
    return primitives


_MOTION_PRIMITIVES = _generate_motion_primitives()


class HybridAStarPlanner(BasePlanner):
    """
    Hybrid A* planner for kinematically feasible path planning.

    Plans in continuous (x, y, θ) space while checking collisions
    on the discrete occupancy grid. Produces smooth, executable paths
    for the differential-drive hospital robot.

    Configuration:
        - Motion primitives: 7 steering angles from -π/4 to +π/4
        - Step size: 1.0 grid cell per motion
        - Theta discretization: 0.4 rad (~23°) bins

    Limitations:
        - Does not model exact robot footprint (uses point collision)
        - Simplified bicycle model (not full Ackermann)
        - No reverse motion (forward-only primitives)

    Attributes:
        _open_set:       Set of grid cells in frontier (for visualization).
        _closed_set:     Set of expanded grid cells.
        _nodes_expanded: Count.
        _nodes_generated: Count.
        _open_peak:      Peak frontier size.
        _XY_RES:         Spatial discretization (cells).
        _THETA_RES:      Angular discretization (radians).
    """

    _XY_RES:    float = 1.0
    _THETA_RES: float = math.pi / 8   # 22.5° bins

    def __init__(
        self,
        occupancy_grid: OccupancyGrid,
        heuristic_type: HeuristicType = HeuristicType.EUCLIDEAN,
        heuristic_weight: float = 1.0,
        max_iterations: int = BasePlanner.DEFAULT_MAX_ITER,
        use_8_connected: bool = True,
        step_size: float = 1.0,
    ) -> None:
        super().__init__(
            occupancy_grid=occupancy_grid,
            heuristic_type=heuristic_type,
            heuristic_weight=heuristic_weight,
            max_iterations=max_iterations,
            use_8_connected=use_8_connected,
        )
        self._step_size = step_size
        self._motion_primitives = _generate_motion_primitives(step_size)

        self._open_set:        Set[GridPos] = set()
        self._closed_set:      Set[GridPos] = set()
        self._closed_keys:     Set[Tuple]   = set()
        self._nodes_expanded:  int = 0
        self._nodes_generated: int = 0
        self._open_peak:       int = 0

    # ------------------------------------------------------------------
    # BasePlanner interface
    # ------------------------------------------------------------------

    @property
    def name(self) -> str:
        return "Hybrid A*"

    def reset(self) -> None:
        self._open_set.clear()
        self._closed_set.clear()
        self._closed_keys.clear()
        self._nodes_expanded  = 0
        self._nodes_generated = 0
        self._open_peak       = 0

    def get_open_set(self)        -> Set[GridPos]: return set(self._open_set)
    def get_closed_set(self)      -> Set[GridPos]: return set(self._closed_set)
    def get_nodes_expanded(self)  -> int: return self._nodes_expanded
```

```
    def get_nodes_generated(self) -> int: return self._nodes_generated
    def get_open_list_peak(self)  -> int: return self._open_peak

    # ──────────────────────────────────────────────────────────────────────
    # Core Hybrid A* Implementation
    # ──────────────────────────────────────────────────────────────────────

    def _plan_impl(self, start: GridPos, goal: GridPos) -> List[GridPos]:
```

FILE: navigation/route_planner.py [Lines 1–160 of 183]

```
"""
Hospital Route Planner — Doorway-Safe Corridor Navigation (Zero Wall Clipping)
================================================================================
Implements a topological-metric path planner:
1. Room Exit: If AMR is currently inside a room, navigates safely out through its door.
2. Corridor Highway Transit: Plans along open central corridor spines (H1, H2, H3, V1, V2, V3)
   using 8-connected A* search with safety inflation.
3. Room Entry: Approaches the target room's exterior corridor waypoint, crosses the doorway
   threshold, and parks at the interior target destination or bedside.

Guarantees 100% door-only room ingress/egress. Zero wall crossing.
"""

from __future__ import annotations

from typing import Any, List, Optional, Tuple
import numpy as np


def plan_hospital_route(
    start_pos: np.ndarray | Tuple[float, float, float] | Tuple[float, float],
    dest_room_id: str,
    hospital_env: Any,
    collision_guard: Any,
    dynamic_obstacles: Optional[List[Tuple[float, float]]] = None,
) -> List[np.ndarray]:
    """
    Plan a 100% collision-free hospital path respecting doors, corridors, and dynamic obstacles.

    Args:
        start_pos: (X, Y, Z) or (X, Y) current position of the AMR.
        dest_room_id: Room ID or bedside marker (e.g. "ICU-01", "PHM-01", "PAT-101").
        hospital_env: HospitalEnvironment instance containing registered semantic locations.
        collision_guard: CollisionGuard instance with wall boundary boxes.
        dynamic_obstacles: Optional list of (X, Y) dynamic obstacle coordinates.

    Returns:
        List of [X, Y, Z] waypoints in world meters.
        Returns empty list [] if no collision-free path exists.
    """
    semantic_locs = hospital_env.get_semantic_locations()
    if dest_room_id not in semantic_locs:
        # Check upper-case or fallback alias check
        upper_dest = dest_room_id.upper().strip()
        if upper_dest in semantic_locs:
            dest_room_id = upper_dest
        else:
            dest_room_id = "PHM-01"

    dest_info = semantic_locs[dest_room_id]
    dest_door = dest_info["door"]
    dest_corridor = dest_info["corridor_wp"]
    dest_inside = dest_info["pos"]

    start_xy = (float(start_pos[0]), float(start_pos[1]))
    route_waypoints: List[Tuple[float, float]] = [start_xy]

    # 1. Determine if start position is currently inside any clinical room (inset by 0.35m to exclude
        doorway/threshold)
    current_room = None
    current_room_id = None
    for r_id, r_info in semantic_locs.items():
        if "bounds" in r_info:
            b_xmin, b_xmax, b_ymin, b_ymax = r_info["bounds"]
            if (b_xmin + 0.35) <= start_xy[0] <= (b_xmax - 0.35) and (b_ymin + 0.35) <= start_xy[1] <= (b_ymax - 0.35)
                :
                current_room = r_info
                current_room_id = r_id
                break
        else:
            rx, ry = float(r_info["pos"][0]), float(r_info["pos"][1])
```

```
            if np.hypot(start_xy[0] - rx, start_xy[1] - ry) < 2.5:
                current_room = r_info
                current_room_id = r_id
                break

    # If inside a room and not already at destination room, route out through its door
    if current_room is not None and current_room_id != dest_room_id:
        door_pt = (float(current_room["door"][0]), float(current_room["door"][1]))
        corr_pt = (float(current_room["corridor_wp"][0]), float(current_room["corridor_wp"][1]))
        if np.hypot(start_xy[0] - door_pt[0], start_xy[1] - door_pt[1]) > 0.35:
            # Check if direct line to door is clear; if not, plan A* to door
            if collision_guard and hasattr(collision_guard, "is_segment_clear") and not
                collision_guard.is_segment_clear(
                start_xy[0], start_xy[1], door_pt[0], door_pt[1], margin=0.30, step_size=0.05
            ):
                exit_to_door = collision_guard.plan_path_astar(start_xy, door_pt)
                if exit_to_door and len(exit_to_door) >= 2:
                    for pt in exit_to_door[1:]:
                        if np.hypot(pt[0] - route_waypoints[-1][0], pt[1] - route_waypoints[-1][1]) > 0.25:
                            route_waypoints.append((float(pt[0]), float(pt[1])))
                else:
                    route_waypoints.append(door_pt)
            else:
                route_waypoints.append(door_pt)

        if np.hypot(door_pt[0] - corr_pt[0], door_pt[1] - corr_pt[1]) > 0.35:
            route_waypoints.append(corr_pt)
        nav_start = corr_pt
    else:
        nav_start = start_xy

    # 2. A* path along open corridor highway from nav_start to target room corridor waypoint
    goal_corr = (float(dest_corridor[0]), float(dest_corridor[1]))

    # Merge dynamic hazards and decaying congestion hotspots from SharedWorldModel
    active_dyn_obs = list(dynamic_obstacles) if dynamic_obstacles else []
    try:
        from world_model.shared_world_model import SHARED_WORLD_MODEL
        congestion_obs = SHARED_WORLD_MODEL.get_dynamic_obstacles_with_congestion(min_penalty_threshold=2.5)
        for c_pt in congestion_obs:
            if not any(np.hypot(c_pt[0] - o[0], c_pt[1] - o[1]) < 0.5 for o in active_dyn_obs):
                active_dyn_obs.append(c_pt)
    except Exception:
        pass

    corridor_path = []
    if active_dyn_obs and len(active_dyn_obs) > 0 and hasattr(collision_guard, "plan_path_astar_dynamic"):
        # Tier 1: Try with safe clinical clearance margin (0.68m)
        corridor_path = collision_guard.plan_path_astar_dynamic(
            nav_start, goal_corr, dynamic_obstacles=active_dyn_obs, obstacle_radius=0.68
        )
        if not corridor_path:
            # Tier 2: Try with tighter clearance margin (0.48m) to squeeze through passable corridor space
            corridor_path = collision_guard.plan_path_astar_dynamic(
                nav_start, goal_corr, dynamic_obstacles=active_dyn_obs, obstacle_radius=0.48
            )

    # Tier 3: If dynamic search fails, fallback to static topological corridor A*
    if not corridor_path:
        corridor_path = collision_guard.plan_path_astar(nav_start, goal_corr)

    # Tier 4: If direct corridor path is still blocked, detour via the 5.0m Central Grand Spine
    if not corridor_path and np.hypot(nav_start[0] - goal_corr[0], nav_start[1] - goal_corr[1]) > 4.0:
        spine_pt = (0.0, 18.0) if nav_start[1] > 5.0 else (0.0, -4.5)
        path1 = collision_guard.plan_path_astar(nav_start, spine_pt)
        path2 = collision_guard.plan_path_astar(spine_pt, goal_corr)
        if path1 and path2:
            corridor_path = path1[:-1] + path2

    if corridor_path and len(corridor_path) >= 2:
        for pt in corridor_path:
            # Avoid duplicate adjacent waypoints
            if not route_waypoints or np.hypot(pt[0] - route_waypoints[-1][0], pt[1] - route_waypoints[-1][1]) > 0.35:
                route_waypoints.append((float(pt[0]), float(pt[1])))
    elif np.hypot(nav_start[0] - goal_corr[0], nav_start[1] - goal_corr[1]) < 0.60:
        # Already at destination corridor waypoint
        if not route_waypoints or np.hypot(goal_corr[0] - route_waypoints[-1][0], goal_corr[1] - route_waypoints[-1][
            1]) > 0.35:
            route_waypoints.append(goal_corr)
    else:
        # Corridor path search failed or completely blocked by obstacles
        # Strictly return empty list to prevent wall clipping
        return []
```

```
    # 3. Enter destination room strictly through its doorway threshold to validated goal
    try:
        from navigation.goal_validator import validate_navigation_goal
    except (ImportError, ModuleNotFoundError):
        from isaac_sim_hospital.navigation.goal_validator import validate_navigation_goal

    val_goal, is_valid, val_msg = validate_navigation_goal(
        dest_room_id, hospital_env, collision_guard, robot_radius=0.38
    )
```

MODULE 7: SLAM AND MAPPING

FILE: environment/occupancy_grid.py [Lines 50–210 of 409]

```
    (-1,  0, 1.0),
    ( 1,  0, 1.0),
    ( 0, -1, 1.0),
    ( 0,  1, 1.0),
]


class OccupancyGrid:
    """
    Dynamic occupancy grid for path planning.

    Maintains:
        _static_cost   — immutable base cost from hospital map
        _dynamic_cost  — mutable overlay for dynamic obstacles
        _dynamic_cells — set of (row, col) currently occupied dynamically

    The effective cost at (r, c) = max(_static_cost[r,c], _dynamic_cost[r,c])
    """

    def __init__(self, hospital_map: HospitalMap) -> None:
        """
        Initialize from a fully built HospitalMap.

        Args:
            hospital_map: Built HospitalMap instance.
        """
        assert hospital_map._built, "HospitalMap must be built before use."

        self._map = hospital_map
        self.rows  = hospital_map.rows
        self.cols  = hospital_map.cols

        # Static cost layer (from map — never changes)
        self._static_cost: np.ndarray = hospital_map.cost_grid.copy()

        # Dynamic cost overlay (modified at runtime)
        self._dynamic_cost: np.ndarray = np.zeros(
            (self.rows, self.cols), dtype=np.float32
        )

        # Set of cells occupied by dynamic obstacles
        self._dynamic_cells: Set[GridPos] = set()

        # Inflation radius (cells) for obstacle proximity penalty
        self._inflation_radius: float = HOSPITAL_CONFIG.ROBOT_SAFE_DISTANCE

        log.info(
            "OccupancyGrid initialized: %d×%d, inflation_radius=%.1f",
            self.rows, self.cols, self._inflation_radius,
        )

    # ──────────────────────────────────────────────────────────────────
    # Core Queries
    # ──────────────────────────────────────────────────────────────────

    def is_free(self, row: int, col: int) -> bool:
        """
        Return True if cell (row, col) is walkable (not wall, not dynamic obstacle).

        Args:
            row: Grid row.
            col: Grid column.

        Returns:
            True if the cell can be traversed.
        """
        if not self._in_bounds(row, col):
            return False
        if self._static_cost[row, col] >= COST_CONFIG.COST_WALL:
            return False
        if (row, col) in self._dynamic_cells:
            return False
        return True

    def get_cost(self, row: int, col: int) -> float:
```

```
        """
        Return the effective traversal cost of cell (row, col).

        Returns:
            Cost in [1.0, ∞). Returns COST_WALL for impassable cells.
        """
        if not self._in_bounds(row, col):
            return COST_CONFIG.COST_WALL
        if (row, col) in self._dynamic_cells:
            return COST_CONFIG.COST_WALL

        base = float(self._static_cost[row, col])
        overlay = float(self._dynamic_cost[row, col])
        return max(base, base + overlay)

    def get_edge_cost(
        self,
        from_pos: GridPos,
        to_pos: GridPos,
    ) -> float:
        """
        Compute the cost of moving from `from_pos` to `to_pos`.

        For diagonal moves, applies the √2 multiplier.
        Returns COST_WALL if destination is not free.

        Args:
            from_pos: Source (row, col).
            to_pos:   Destination (row, col).

        Returns:
            Edge traversal cost.
        """
        if not self.is_free(*to_pos):
            return COST_CONFIG.COST_WALL

        dr = abs(to_pos[0] - from_pos[0])
        dc = abs(to_pos[1] - from_pos[1])
        move_cost = 1.414 if (dr + dc == 2) else 1.0

        return move_cost * self.get_cost(*to_pos)

    # ──────────────────────────────────────────────────────────────────────
    # Neighbor Expansion (used by all planners)
    # ──────────────────────────────────────────────────────────────────────

    def get_neighbors_4(self, row: int, col: int) -> List[Tuple[GridPos, float]]:
        """
        Return 4-connected walkable neighbors with edge costs.

        Returns:
            List of ((r, c), cost) tuples.
        """
        result = []
        for dr, dc, move_cost in _MOVES_4:
            nr, nc = row + dr, col + dc
            if self.is_free(nr, nc):
                cost = move_cost * self.get_cost(nr, nc)
                result.append(((nr, nc), cost))
        return result

    def get_neighbors_8(self, row: int, col: int) -> List[Tuple[GridPos, float]]:
        """
        Return 8-connected walkable neighbors with edge costs.

        Diagonal moves are blocked if both cardinal neighbors are walls
        (prevents cutting corners through walls).

        Returns:
            List of ((r, c), cost) tuples.
        """
        result = []
        for dr, dc, move_cost in _MOVES_8:
            nr, nc = row + dr, col + dc
            if not self.is_free(nr, nc):
                continue

            # Corner-cutting check for diagonal moves
            if dr != 0 and dc != 0:
                if (not self.is_free(row + dr, col) or
                        not self.is_free(row, col + dc)):
                    continue   # Block diagonal through wall corner
```

```
            cost = move_cost * self.get_cost(nr, nc)
            result.append(((nr, nc), cost))
        return result
```

FILE: isaac_sim_hospital/navigation/slam_exploration_engine.py [Lines 30–165 of 1063]

```
class OccupancyState:
    UNKNOWN = -1
    FREE = 0
    OCCUPIED = 100
    DYNAMIC_OBSTACLE = 90


@dataclass
class FrontierCluster:
    centroid: Tuple[float, float]
    cells: List[Tuple[int, int]]
    size: int
    utility: float = 0.0

    @property
    def x(self) -> float:
        return self.centroid[0]

    @property
    def y(self) -> float:
        return self.centroid[1]


class SLAMExplorationEngine:
    """
    Real-time 2D SLAM Occupancy Grid Mapper, Frontier Exploration Planner,
    and Dynamic Obstacle Life-Cycle Tracker for AAROGYA AMRs.
    """

    def __init__(
        self,
        grid_bounds: Tuple[float, float, float, float] = (-46.0, 46.0, -40.0, 40.0),
        resolution: float = 0.5,
        max_lidar_range: float = 12.0,
        inflation_radius: float = 0.65,
    ) -> None:
        """
        Args:
            grid_bounds: (xmin, xmax, ymin, ymax) in meters.
            resolution: Grid cell size in meters (0.5m matches ROS 2 /map).
            max_lidar_range: Maximum usable range for ray integration.
            inflation_radius: Safe collision dilation in meters.
        """
        self.xmin, self.xmax, self.ymin, self.ymax = grid_bounds
        self.resolution = resolution
        self.max_lidar_range = max_lidar_range
        self.inflation_radius = inflation_radius
        self.inflation_cells = int(math.ceil(self.inflation_radius / self.resolution))

        self.cols = int(round((self.xmax - self.xmin) / self.resolution))
        self.rows = int(round((self.ymax - self.ymin) / self.resolution))

        # 1. Live SLAM Occupancy Grid: -1 = UNKNOWN, 0 = FREE, 100 = OCCUPIED
        self.grid = np.full((self.rows, self.cols), OccupancyState.UNKNOWN, dtype=np.int8)

        # 2. Dynamic Obstacle Layer: 0 = Clear, 100 = Dynamic Obstacle
        self.dynamic_layer = np.zeros((self.rows, self.cols), dtype=np.uint8)

        # 3. Observation Timestamps & Hit Counts
        self.last_observed = np.zeros((self.rows, self.cols), dtype=np.float32)
        self.hit_counts = np.zeros((self.rows, self.cols), dtype=np.uint16)
        self.pass_counts = np.zeros((self.rows, self.cols), dtype=np.uint16)

        # 4. Active Dynamic Obstacles Tracker: id -> dict
        self.active_dynamic_obstacles: Dict[str, Dict[str, Any]] = {}

        # 5. Frontier state cache
        self.frontiers: List[FrontierCluster] = []
        self.last_frontier_calc_time: float = 0.0
        self.exploration_complete: bool = False

        # 6. Unreachable frontier tracking with TTL to prevent infinite retry lockouts
        self.unreachable_frontiers: Dict[Tuple[float, float], float] = {}

        # Reference corridor footprint cell count (hospital navigable area ~ 4,200 cells)
        self.expected_navigable_cells: int = 4200
```

```python
    def world_to_grid(self, x: float, y: float) -> Tuple[int, int]:
        """Convert world coordinates (X, Y) to grid indices (row, col)."""
        col = int(round((x - self.xmin) / self.resolution))
        row = int(round((self.ymax - y) / self.resolution))
        col = max(0, min(self.cols - 1, col))
        row = max(0, min(self.rows - 1, row))
        return row, col

    def grid_to_world(self, row: int, col: int) -> Tuple[float, float]:
        """Convert grid cell (row, col) to world coordinates (X, Y) in meters."""
        x = self.xmin + (col + 0.5) * self.resolution
        y = self.ymax - (row + 0.5) * self.resolution
        return round(x, 2), round(y, 2)

    def is_in_bounds(self, row: int, col: int) -> bool:
        return 0 <= row < self.rows and 0 <= col < self.cols

    # ── 1. REAL-TIME LIDAR RAYCAST INTEGRATION (SLAM MAPPING) ──────────────

    def integrate_lidar_scan(
        self,
        robot_x: float,
        robot_y: float,
        robot_yaw: float,
        scan: Any = None,
        dynamic_agents: Optional[List[Any]] = None,
        **kwargs: Any,
    ) -> Dict[str, Any]:
        """
        Integrates a real 2D planar LiDAR scan from the robot's current pose.
        - Traces rays using Bresenham vector steps.
        - Carves out FREE space along the beam.
        - Marks endpoints as OCCUPIED.
        - Detects dynamic obstacle placement and clears removed obstacles.
        """
        if scan is None:
            scan = kwargs.get("scan_result")
        now = time.time()
        start_r, start_c = self.world_to_grid(robot_x, robot_y)
        # Robot's immediate footprint is unconditionally FREE
        for dr in range(-1, 2):
            for dc in range(-1, 2):
                nr, nc = start_r + dr, start_c + dc
                if self.is_in_bounds(nr, nc):
                    self.grid[nr, nc] = OccupancyState.FREE
                    self.dynamic_layer[nr, nc] = 0

        ranges = getattr(scan, "ranges", getattr(scan, "distances_m", []))
        angles = getattr(scan, "angles", getattr(scan, "angles_rad", None))
        if angles is None and hasattr(scan, "angles_deg"):
            angles = np.radians(getattr(scan, "angles_deg"))
        elif angles is None:
            angles = []
        tags = getattr(scan, "tags", getattr(scan, "obstacle_tags", []))

        num_beams = min(len(ranges), len(angles))
        if num_beams == 0:
            return self.get_exploration_progress()
```

MODULE 8: HOSPITAL MANAGEMENT AND DATABASE

FILE: database/models.py [Lines 35–220 of 443]

```python
class Base(DeclarativeBase):
    pass


# ── ENUMS ──────────────────────────────────────────────────────────────

class UserRole(str, enum.Enum):
    ADMIN = "ADMIN"
    DOCTOR = "DOCTOR"
    NURSE = "NURSE"
    PATIENT = "PATIENT"
    VISITOR = "VISITOR"


class PatientStatusEnum(str, enum.Enum):
    ACTIVE = "ACTIVE"          # Under treatment
    WAITING = "WAITING"        # Triage / waiting
    EMERGENCY = "EMERGENCY"    # Critical care / Code Blue
    OUTPATIENT = "OUTPATIENT"  # Day care
    DISCHARGED = "DISCHARGED"  # Released
    INACTIVE = "INACTIVE"


class PrescriptionStatusEnum(str, enum.Enum):
    ACTIVE = "ACTIVE"
    PAUSED = "PAUSED"
    DISCONTINUED = "DISCONTINUED"
    COMPLETED = "COMPLETED"
    EXPIRED = "EXPIRED"


class ScheduleStatusEnum(str, enum.Enum):
    PENDING = "PENDING"
    ASSIGNED = "ASSIGNED"
    IN_TRANSIT = "IN_TRANSIT"
    DELIVERED = "DELIVERED"
    MISSED = "MISSED"
    CANCELLED = "CANCELLED"


class TaskPriorityEnum(str, enum.Enum):
    LOW = "LOW"
    NORMAL = "NORMAL"
    HIGH = "HIGH"
    EMERGENCY = "EMERGENCY"


class TaskTypeEnum(str, enum.Enum):
    MEDICINE_DELIVERY = "MEDICINE_DELIVERY"
    PATIENT_ESCORT = "PATIENT_ESCORT"
    EMERGENCY_RESPONSE = "EMERGENCY_RESPONSE"
    SUPPLY_FETCH = "SUPPLY_FETCH"
    ROOM_GUIDANCE = "ROOM_GUIDANCE"


class TaskStatusEnum(str, enum.Enum):
    PENDING = "PENDING"
    VALIDATING = "VALIDATING"
    APPROVED = "APPROVED"
    ASSIGNED = "ASSIGNED"
    IN_PROGRESS = "IN_PROGRESS"
    ARRIVED = "ARRIVED"
    DELIVERED = "DELIVERED"
    VERIFIED = "VERIFIED"
    FAILED = "FAILED"
    CANCELLED = "CANCELLED"


class RobotRoleEnum(str, enum.Enum):
    DELIVERY = "DELIVERY"
    RECEPTION = "RECEPTION"
    EMERGENCY = "EMERGENCY"
    SUPPORT = "SUPPORT"
```

```python
class RobotStatusEnum(str, enum.Enum):
    IDLE = "IDLE"
    PLANNING = "PLANNING"
    NAVIGATING = "NAVIGATING"
    EXECUTING = "EXECUTING"
    CHARGING = "CHARGING"
    MAINTENANCE = "MAINTENANCE"
    OFFLINE = "OFFLINE"


# ── RELATIONAL ENTITIES ──────────────────────────────────────────

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    username: Mapped[str] = mapped_column(String(64), unique=True, nullable=False, index=True)
    password_hash: Mapped[str] = mapped_column(String(256), nullable=False)
    full_name: Mapped[str] = mapped_column(String(128), nullable=False)
    role: Mapped[UserRole] = mapped_column(Enum(UserRole), nullable=False, default=UserRole.VISITOR, index=True)
    email: Mapped[Optional[str]] = mapped_column(String(128), unique=True, nullable=True)
    phone: Mapped[Optional[str]] = mapped_column(String(32), nullable=True)
    is_active: Mapped[bool] = mapped_column(Boolean, default=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=utc_now)

    # Profiles
    doctor_profile: Mapped[Optional[Doctor]] = relationship("Doctor", back_populates="user", uselist=False)
    nurse_profile: Mapped[Optional[Nurse]] = relationship("Nurse", back_populates="user", uselist=False)
    patient_profile: Mapped[Optional[Patient]] = relationship("Patient", back_populates="user", uselist=False)
    audit_logs: Mapped[List[AuditLog]] = relationship("AuditLog", back_populates="user")


class Department(Base):
    __tablename__ = "departments"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    name: Mapped[str] = mapped_column(String(64), unique=True, nullable=False)
    code: Mapped[str] = mapped_column(String(16), unique=True, nullable=False, index=True)
    floor: Mapped[int] = mapped_column(Integer, default=1)
    description: Mapped[Optional[str]] = mapped_column(String(256), nullable=True)
    is_restricted: Mapped[bool] = mapped_column(Boolean, default=False)

    rooms: Mapped[List[HospitalRoom]] = relationship("HospitalRoom", back_populates="department")
    doctors: Mapped[List[Doctor]] = relationship("Doctor", back_populates="department")
    nurses: Mapped[List[Nurse]] = relationship("Nurse", back_populates="department")


class HospitalRoom(Base):
    __tablename__ = "hospital_rooms"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    room_number: Mapped[str] = mapped_column(String(32), unique=True, nullable=False, index=True)
    name: Mapped[str] = mapped_column(String(64), nullable=False)
    department_id: Mapped[Optional[int]] = mapped_column(Integer, ForeignKey("departments.id"), nullable=True)
    zone: Mapped[str] = mapped_column(String(32), default="General")
    x_coord: Mapped[float] = mapped_column(Float, default=0.0)
    y_coord: Mapped[float] = mapped_column(Float, default=0.0)
    z_coord: Mapped[float] = mapped_column(Float, default=0.0)
    approach_yaw: Mapped[float] = mapped_column(Float, default=0.0)
    is_occupied: Mapped[bool] = mapped_column(Boolean, default=False)

    department: Mapped[Optional[Department]] = relationship("Department", back_populates="rooms")
    beds: Mapped[List[Bed]] = relationship("Bed", back_populates="room")
    patients: Mapped[List[Patient]] = relationship("Patient", back_populates="room")


class Bed(Base):
    __tablename__ = "beds"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    bed_number: Mapped[str] = mapped_column(String(32), nullable=False, index=True)
    room_id: Mapped[int] = mapped_column(Integer, ForeignKey("hospital_rooms.id"), nullable=False)
    is_occupied: Mapped[bool] = mapped_column(Boolean, default=False)

    room: Mapped[HospitalRoom] = relationship("HospitalRoom", back_populates="beds")
    patients: Mapped[List[Patient]] = relationship("Patient", back_populates="bed")


class Doctor(Base):
    __tablename__ = "doctors"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(Integer, ForeignKey("users.id"), unique=True, nullable=False)
```

```
    staff_id: Mapped[str] = mapped_column(String(32), unique=True, nullable=False, index=True)
    specialization: Mapped[str] = mapped_column(String(64), nullable=False)
    department_id: Mapped[Optional[int]] = mapped_column(Integer, ForeignKey("departments.id"), nullable=True)
    is_available: Mapped[bool] = mapped_column(Boolean, default=True)

    user: Mapped[User] = relationship("User", back_populates="doctor_profile")
    department: Mapped[Optional[Department]] = relationship("Department", back_populates="doctors")
    patients: Mapped[List[Patient]] = relationship("Patient", back_populates="attending_doctor")
    prescriptions: Mapped[List[Prescription]] = relationship("Prescription", back_populates="doctor")


class Nurse(Base):
    __tablename__ = "nurses"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id: Mapped[int] = mapped_column(Integer, ForeignKey("users.id"), unique=True, nullable=False)
    staff_id: Mapped[str] = mapped_column(String(32), unique=True, nullable=False, index=True)
    department_id: Mapped[Optional[int]] = mapped_column(Integer, ForeignKey("departments.id"), nullable=True)
    shift: Mapped[str] = mapped_column(String(32), default="Morning")
    is_on_duty: Mapped[bool] = mapped_column(Boolean, default=True)

    user: Mapped[User] = relationship("User", back_populates="nurse_profile")
    department: Mapped[Optional[Department]] = relationship("Department", back_populates="nurses")


class Patient(Base):
    __tablename__ = "patients"
```

FILE: database/db_query_engine.py [Lines 35–145 of 383]

```
class DBQueryEngine:
    """High-level query facade for voice/NLU commands.

    All methods return dictionaries suitable for response generation —
    the caller never touches raw SQL or the database directly.
    """

    def __init__(self, db: HospitalDatabase) -> None:
        self._db = db

    # ── Patient ID Resolution ──────────────────────────────────────

    @staticmethod
    def resolve_patient_id(raw: str) -> Optional[str]:
        """Normalize various patient ID formats to canonical form (e.g., P001).

        Accepts: "P001", "P1", "p003", "patient 3", "patient P003", "003"
        Returns: "P001" .. "P006", or None if unresolvable.
        """
        if not raw:
            return None
        text = raw.strip().upper()

        # Direct match: P001 .. P999
        m = re.match(r"^P\s*0*(\d{1,3})$", text)
        if m:
            return f"P{int(m.group(1)):03d}"

        # "PATIENT 3", "PATIENT P003", "PATIENT P3", "PATIENT #2", "PATIENT NO. 2"
        m = re.search(r"PATIENT\s*(?:#|NO\.?)?\s*P?\s*0*(\d{1,3})", text)
        if m:
            return f"P{int(m.group(1)):03d}"

        # Bare number: "3", "03", "003"
        m = re.match(r"^0*(\d{1,3})$", text)
        if m:
            num = int(m.group(1))
            if 1 <= num <= 999:
                return f"P{num:03d}"

        return None

    # ── Patient Queries ────────────────────────────────────────────

    def get_patient_info(self, patient_id: str) -> Optional[Dict[str, Any]]:
        """Full patient info including status, doctor, room, condition."""
        pid = self.resolve_patient_id(patient_id) or patient_id
        patient = self._db.get_patient(pid)
        if not patient:
            return None

        status = self._db.get_patient_status(pid)
```

```
        doctor = self._db.get_attending_doctor(pid)
        encounter = self._db.get_current_encounter(pid)
        prescriptions = self._db.get_prescriptions(pid)

        return {
            "patient_id": pid,
            "name": patient["name"],
            "age": patient.get("age"),
            "gender": patient.get("gender"),
            "room_id": patient.get("room_id"),
            "assigned_room": patient.get("room_id"),
            "condition": patient.get("condition"),
            "condition_summary": patient.get("condition"),
            "priority": patient.get("priority"),
            "status": status["status"] if status else "UNKNOWN",
            "live_status": status["status"] if status else "UNKNOWN",
            "status_updated_at": status.get("updated_at") if status else None,
            "doctor_name": doctor["name"] if doctor else "Unassigned",
            "attending_doctor": doctor["name"] if doctor else "Unassigned",
            "attending_doctor_name": doctor["name"] if doctor else "Unassigned",
            "doctor_specialization": doctor.get("specialization") if doctor else None,
            "doctor_id": doctor["doctor_id"] if doctor else None,
            "treatment": encounter.get("current_treatment") if encounter else None,
            "current_treatment": encounter.get("current_treatment") if encounter else None,
            "encounter": {
                "admission_time": encounter.get("admission_time"),
                "treatment": encounter.get("current_treatment"),
                "condition": encounter.get("current_condition"),
            } if encounter else None,
            "prescriptions": [
                {
                    "medicine": rx["medicine_name"],
                    "medicine_name": rx["medicine_name"],
                    "dosage": rx["dosage"],
                    "frequency": rx["frequency"],
                    "route": rx["route"],
                }
                for rx in prescriptions
            ],
        }

    def get_patient_status_brief(self, patient_id: str) -> Optional[Dict[str, Any]]:
        """Brief status check suitable for voice response."""
        pid = self.resolve_patient_id(patient_id) or patient_id
        status = self._db.get_patient_status(pid)
        if not status:
            return None
        return {
            "patient_id": pid,
            "name": status["name"],
            "status": status["status"],
            "room_id": status["room_id"],
            "condition": status["condition"],
            "doctor_name": status.get("doctor_name", "Unassigned"),
        }

    def get_attending_doctor_info(self, patient_id: str) -> Optional[Dict[str, Any]]:
```

MODULE 9: FRONTEND AND UI INTEGRATION

FILE: frontend/static/js/app.js [Lines 1–150 of 3340]

```
/**
 * AAROGYA — Hospital Autonomous Assistance Platform
 * Frontend Master Controller & Isaac Sim Realtime Dashboard
 */

// ── GLOBAL APPLICATION CONTROLLER ──────────────────────────────────
const AarogyaApp = (function () {
  // Application State
  const state = {
    token: localStorage.getItem("aarogya_token") || null,
    user: JSON.parse(localStorage.getItem("aarogya_user") || "null"),
    currentView: "dashboard",
    simRunning: false,
    cameraMode: "FOLLOW_ROBOT",
    activeRobotId: "Arogya_Delivery",
    robots: {},
    tasks: [],
    patients: [],
    prescriptions: [],
    medicines: [],
    doctors: [],
    nurses: [],
    auditLogs: [],
    emergencies: [],
    rooms: [
      { code: "CHG-01", name: "Charging Bay 1", zone: "Fleet", x: 37.25, y: 26.0, w: 10, h: 8, color: "rgba(6, 182,
          212, 0.2)" },
      { code: "RCT-01", name: "Reception Atrium", zone: "Public", x: 0.0, y: -5.0, w: 16, h: 12, color: "rgba(59,
          130, 246, 0.2)" },
      { code: "PHM-01", name: "Pharmacy Station", zone: "Clinical", x: 14.0, y: 23.2, w: 12, h: 10, color: "rgba(16,
          185, 129, 0.25)" },
      { code: "ICU-01", name: "Critical Care Unit A", zone: "Restricted", x: -36.5, y: 11.0, w: 14, h: 12, color:
          "rgba(244, 63, 94, 0.25)", beds: ["ICU Bed 1", "ICU Bed 2", "ICU Bed 3"] },
      { code: "EMR-01", name: "Trauma Staging", zone: "Emergency", x: -35.0, y: -26.0, w: 14, h: 10, color: "rgba(
          239, 68, 68, 0.3)" },
      { code: "OT-01", name: "Operating Theater", zone: "Surgical", x: -39.0, y: 26.0, w: 12, h: 10, color: "rgba(
          139, 92, 246, 0.2)" },
      { code: "LAB-01", name: "Pathology Lab", zone: "Diagnostics", x: -0.5, y: 26.0, w: 11, h: 9, color: "rgba(245,
          158, 11, 0.2)" },
      { code: "RAD-01", name: "Radiology Suite", zone: "Diagnostics", x: -15.5, y: 26.0, w: 12, h: 9, color: "rgba(
          168, 85, 247, 0.2)" },
      { code: "OPD-01", name: "Outpatient Desk", zone: "Public", x: 24.0, y: -26.0, w: 14, h: 10, color: "rgba(59,
          130, 246, 0.15)" },
      { code: "PAT-101", name: "Ward Room 101", zone: "Inpatient", x: 10.0, y: -15.0, w: 10, h: 8, color: "rgba(16,
          185, 129, 0.15)", beds: ["Bed 101-A", "Bed 101-B"] },
      { code: "PAT-102", name: "Ward Room 102", zone: "Inpatient", x: 18.0, y: -15.0, w: 10, h: 8, color: "rgba(16,
          185, 129, 0.15)", beds: ["Bed 102-A"] },
    ],
    ws: null,
    speechRecognition: null,
    isListening: false,
    lastLidarScan: null,
    slamMap: null,
    slamProgress: null,
    activeDynamicObstacles: [],
    mapViewMode: "reference", // "reference", "slam", "side", "overlay"
    overlayOpacity: 0.5,
    selectedFleetRobotId: "Arogya_Delivery",
    fleetStatusData: null,
    showSemantics: true,
    authoritativeKnowledgeMap: null,
    replayMissionData: null,
    replayAnimFrameId: null,
    replayCurrentStep: 0,
    replayIsPlaying: false,
  };

  // ── 1. HTTP REQUEST UTILITY ──────────────────────────────────────
  async function apiRequest(endpoint, options = {}) {
    const headers = { "Content-Type": "application/json", ...(options.headers || {}) };
    if (state.token) {
      headers["Authorization"] = `Bearer ${state.token}`;
    }
    const res = await fetch(endpoint, { ...options, headers });
    if (res.status === 401) {
```

```
      showLoginModal();
      throw new Error("Authentication required");
    }
    if (res.status === 409) {
      const conflict = await res.json();
      throw { status: 409, data: conflict };
    }
    if (!res.ok) {
      const err = await res.json().catch(() => ({ detail: res.statusText }));
      throw new Error(err.detail || "API request failed");
    }
    return res.json();
  }

  // ── 2. INITIALIZATION ──────────────────────────────────────────
  async function init() {
    setupNavigation();
    setupModals();
    setupCanvas();
    setupSpeech();
    setupTeleopControls();
    setupZoomControls();
    setupInteractiveMapCanvas("canvas-map-reference", "reference", "cursor-coord-ref");
    setupInteractiveMapCanvas("canvas-map-slam", "slam", "cursor-coord-slam");

    // Default to doctor if not logged in
    if (!state.token || !state.user) {
      await quickLogin("doctor", "doctor123");
    } else {
      updateUserUI();
    }

    // Connect real-time telemetry WebSocket
    connectWebSocket();

    // Initial data load
    await refreshAllData();

    // Check URL query parameters for view routing (e.g. ?view=simulation)
    const urlParams = new URLSearchParams(window.location.search);
    const initialView = urlParams.get("view");
    if (initialView) {
      switchView(initialView);
    }

    // Periodic poll for DB updates
    setInterval(refreshAllData, 4000);

    // Live Hospital Chronometer
    setInterval(() => {
      const el = document.getElementById("header-chronometer");
      if (el) {
        const d = new Date();
        el.textContent = d.toISOString().slice(11, 19) + " UTC // " + d.toLocaleTimeString();
      }
    }, 1000);
  }

  // ── 3. AUTHENTICATION & RBAC ──────────────────────────────────
  async function login(username, password) {
    try {
      const data = await apiRequest("/api/auth/login", {
        method: "POST",
        body: JSON.stringify({ username, password }),
      });
      const u = data.user || data;
      state.token = data.access_token;
      state.user = {
        id: u.id || data.user_id,
        role: u.role || data.role,
        full_name: u.full_name,
        username: u.username,
        email: u.email,
      };
      localStorage.setItem("aarogya_token", state.token);
      localStorage.setItem("aarogya_user", JSON.stringify(state.user));

      updateUserUI();
      closeModal("modal-login");
      showToast(`Logged in as ${data.full_name} (${data.role})`);
      await refreshAllData();
    } catch (err) {
      alert("Login failed: " + (err.message || "Invalid credentials"));
```

```
    }
  }
```

FILE: frontend/index.html [Lines 1–90 of 1281]

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>AAROGYA — Hospital Autonomous Assistance Platform & Robotics Operations Console</title>
  <meta name="description" content="Multi-Agent Embodied AI Hospital Robot Integration and Management Platform with
      NVIDIA Isaac Sim 5.1 — Hospital Autonomous Assistance Platform">
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&family=Outfit:wght@500;600;700;800&f
      amily=JetBrains+Mono:wght@400;500;600;700&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="/static/css/app.css?v=4.0.0">
</head>
<body>
  <div id="app-container">
    <!-- ── COMMAND BRIDGE SIDEBAR NAVIGATION ── -->
    <aside id="sidebar">
      <div class="brand-header">
        <div class="brand-icon">
          <svg width="22" height="22" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2.5"
              stroke-linecap="round" stroke-linejoin="round">
            <path d="M12 2v20M2 12h20M7 7l10 10M17 7L7 17"/>
          </svg>
        </div>
        <div>
          <div class="brand-title">
            AAROGYA
            <span class="brand-version">v5.1</span>
          </div>
          <div class="brand-subtitle">Autonomous Robotics Core</div>
        </div>
      </div>

      <nav class="nav-menu">
        <div class="nav-section-label">Tier 1 // Tactical Control</div>
        <a class="nav-item active" data-view="overview" id="nav-overview">
          <span class="nav-item-num">01</span>
          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor"><rect x="3" y="3" width="7" height="7"/><rect
              x="14" y="3" width="7" height="7"/><rect x="14" y="14" width="7" height="7"/><rect x="3" y="14"
              width="7" height="7"/></svg>
          Command Deck
        </a>
        <a class="nav-item" data-view="simulation" id="nav-simulation">
          <span class="nav-item-num">02</span>
          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor"><circle cx="12" cy="12" r="10"/><polygon
              points="10 8 16 12 10 16 10 8"/></svg>
          Simulation Control
        </a>
        <a class="nav-item" data-view="map" id="nav-map">
          <span class="nav-item-num">03</span>
          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor"><polygon points="1 6 1 22 8 18 16 22 23 18 23 2
              16 6 8 2 1 6"/><line x1="8" y1="2" x2="8" y2="18"/><line x1="16" y1="6" x2="16" y2="22"/></svg>
          Hospital Live Map
        </a>
        <a class="nav-item" data-view="fleet" id="nav-fleet">
          <span class="nav-item-num">04</span>
          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor"><rect x="3" y="11" width="18" height="10"
              rx="2"/><circle cx="12" cy="5" r="2"/><path d="M12 7v4"/><line x1="8" y1="16" x2="8" y2="16"/><line
              x1="16" y1="16" x2="16" y2="16"/></svg>
          Robot Fleet
        </a>
        <a class="nav-item" data-view="tasks" id="nav-tasks">
          <span class="nav-item-num">05</span>
          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor"><polyline points="9 11 12 14 22 4"/><path d="M21
              12v7a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h11"/></svg>
          Missions & Tasks
        </a>

        <div class="nav-section-label">Tier 2 // Clinical Care</div>
        <a class="nav-item" data-view="patients" id="nav-patients">
          <span class="nav-item-num">06</span>
          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor"><path d="M17 21v-2a4 4 0 0 0-4-4H5a4 4 0 0 0-4
              4v2"/><circle cx="9" cy="7" r="4"/><path d="M23 21v-2a4 4 0 0 0-3-3.87"/><path d="M16 3.13a4 4 0 0 1 0
              7.75"/></svg>
          Patients & Beds
        </a>
```

```
<a class="nav-item" data-view="staff" id="nav-staff">
  <span class="nav-item-num">07</span>
  <svg viewBox="0 0 24 24" fill="none" stroke="currentColor"><path d="M16 21v-2a4 4 0 0 0-4-4H6a4 4 0 0 0-4
      4v2"/><circle cx="9" cy="7" r="4"/><path d="M22 21v-2a4 4 0 0 0-3-3.87"/><path d="M16 3.13a4 4 0 0 1 0
      7.75"/></svg>
  Doctors & Nurses
</a>
<a class="nav-item" data-view="pharmacy" id="nav-pharmacy">
  <span class="nav-item-num">08</span>
  <svg viewBox="0 0 24 24" fill="none" stroke="currentColor"><path d="m10.5 20.5 10-10a4.95 4.95 0 1
      0-7-7l-10 10a4.95 4.95 0 1 0 7 7Z"/><path d="m8.5 8.5 7 7"/></svg>
  Pharmacy
</a>

<div class="nav-section-label">Tier 3 // Emergency & Audit</div>
<a class="nav-item" data-view="emergency" id="nav-emergency">
  <span class="nav-item-num">09</span>
  <svg viewBox="0 0 24 24" fill="none" stroke="currentColor"><polygon points="7.86 2 16.14 2 22 7.86 22 16.14
      16.14 22 7.86 22 2 16.14 2 7.86 7.86 2"/><line x1="12" y1="8" x2="12" y2="12"/><line x1="12" y1="16"
      x2="12.01" y2="16"/></svg>
  Emergency Centre
</a>
<a class="nav-item" data-view="health" id="nav-health">
  <span class="nav-item-num">10</span>
  <svg viewBox="0 0 24 24" fill="none" stroke="currentColor"><path d="M22 12h-4l-3 9L9 3l-3 9H2"/></svg>
  System Health
</a>
<a class="nav-item" data-view="logs" id="nav-logs">
  <span class="nav-item-num">11</span>
  <svg viewBox="0 0 24 24" fill="none" stroke="currentColor"><path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2
      2h12a2 2 0 0 0 2-2V8z"/><polyline points="14 2 14 8 20 8"/><line x1="12" y1="18" x2="12" y2="12"/><line
      x1="9" y1="15" x2="15" y2="15"/></svg>
```